# Manual Técnico del Sistema de Gestión de Puentes

Arquitectura, módulos y operación

Equipo de desarrollo SGP

Ángel Abelino Son Vicente — 202030885

Melvin Eduardo Ordoñez Sapón — 202230552

Brigido Josías Alvarado Pec — 202230251

Kevin Alexander Ramírez Mushín — 202231207

Dany Noé de León Santizo — 201830006

Miguel Angel Quemé Gómez — 202031398

Mario Raúl Yancor Ocaña — 201930761

Herberth Julian Reyes Pacajoj — 202230236

Kenny Marco Augusto López Salazar — 202031554

Rony Mauricio Rojas Aguilar — 202031191

Universidad de San Carlos de Guatemala — Centro Universitario de Occidente

Octubre de 2026

# Índice

**1 Introducción y alcance** **1**
1.1 Propósito . . . 1
1.2 Alcance funcional . . . 1
1.3 Límites . . . 1

**2 Tecnologías y dependencias principales** **1**

**3 Requisitos de hardware y software** **2**
3.1 Software . . . 2
3.2 Hardware . . . 3

**4 Instalación y configuración del entorno** **3**
4.1 1. Configurar variables . . . 3
4.2 2. Iniciar PostgreSQL/PostGIS . . . 3
4.3 3. Iniciar la API . . . 3
4.4 4. Iniciar Angular . . . 4
4.5 Alternativa con Compose . . . 4
4.6 Configuración de archivos . . . 4

**5 Base de datos y relaciones principales** **4**
5.0.1 Esquema relacional implementado . . . 4

**6 Estructura del proyecto** **6**
6.1 Backend . . . 6
6.2 Cliente . . . 6
6.2.1 Organización por paquetes . . . 8
6.3 Persistencia y documentación . . . 8

**7 Estrategia y ejecución de pruebas** **8**
7.1 Verificaciones de CI . . . 9
7.2 Verificación de seguridad . . . 9
7.3 Pruebas de flujos funcionales . . . 9

**8 Arquitectura y modelo de datos** **10**
8.1 Descripción . . . 10
8.2 Tecnologías . . . 10
8.3 Funcionamiento . . . 11
8.4 Arquitectura de componentes . . . 11
8.5 Consideraciones técnicas . . . 11

**9 Seguridad y sesión** **14**
9.1 Descripción . . . 14
9.2 Tecnologías . . . 14
9.3 Funcionamiento . . . 14
9.4 Flujo de autenticación y renovación . . . 14
9.5 Consideraciones técnicas . . . 14
9.6 Autorización por rol y alcance del recurso . . . 14

9.7 Pruebas y controles por ambiente . . . . . 16

**10 Usuarios y cursos** **17**
10.1 Descripción . . . . . 17
10.2 Tecnologías . . . . . 17
10.3 Funcionamiento . . . . . 17
10.4 Registro e inscripción académica . . . . . 17
10.5 Consideraciones técnicas . . . . . 17
10.6 Recorridos de alta y habilitación . . . . . 19

**11 Puentes y asignaciones** **19**
11.1 Descripción . . . . . 19
11.2 Tecnologías . . . . . 19
11.3 Funcionamiento . . . . . 20
11.4 Inventario y asignaciones . . . . . 20
11.5 Consideraciones técnicas . . . . . 20
11.6 Privacidad de coordenadas para puentes en estado Malo . . . . . 20

**12 Formularios y calibración** **20**
12.1 Descripción . . . . . 22
12.2 Tecnologías . . . . . 22
12.3 Funcionamiento . . . . . 22
12.4 Publicación y calibración de formularios . . . . . 22
12.5 Consideraciones técnicas . . . . . 22
12.6 Siembra inicial, publicación y aprobación académica . . . . . 22

**13 Índice de condición** **24**
13.1 Descripción . . . . . 24
13.2 Tecnologías . . . . . 24
13.3 Funcionamiento . . . . . 24
13.4 Cálculo y publicación del IC . . . . . 24
13.4.1 Ejemplo numérico . . . . . 24
13.5 Consideraciones técnicas . . . . . 26
13.6 Validaciones del motor y lectura de resultados . . . . . 26

**14 Archivos y evidencias** **26**
14.1 Descripción . . . . . 26
14.2 Tecnologías . . . . . 26
14.3 Funcionamiento . . . . . 26
14.4 Carga y acceso a evidencias . . . . . 27
14.5 Consideraciones técnicas . . . . . 27
14.6 Contrato de carga de una evidencia . . . . . 27

**15 Inspecciones** **29**
15.1 Descripción . . . . . 29
15.2 Tecnologías . . . . . 29
15.3 Funcionamiento . . . . . 30
15.4 Ciclo de vida de la inspección . . . . . 30
15.5 Consideraciones técnicas . . . . . 30

15.6 Consulta histórica, impresión y exportación (HU-072) . . . 30
15.7 Secuencia de captura y cierre . . . 30
15.8 Contratos de API para el recorrido . . . 32

**16 Offline y sincronización** **33**
16.1 Descripción . . . 33
16.2 Tecnologías . . . 33
16.3 Funcionamiento . . . 33
16.4 Captura y sincronización sin conexión . . . 34
16.5 Consideraciones técnicas . . . 34
16.6 Divergencias y copia del servidor (HU-059) . . . 34

**17 Revisión académica** **36**
17.1 Descripción . . . 36
17.2 Tecnologías . . . 36
17.3 Funcionamiento . . . 36
17.4 Ciclo de revisión académica . . . 36
17.5 Consideraciones técnicas . . . 36
17.6 Resultado de un ciclo y diagnóstico . . . 36

**18 Mantenimiento** **38**
18.1 Descripción . . . 38
18.2 Tecnologías . . . 38
18.3 Funcionamiento . . . 38
18.4 Generación y ejecución de mantenimiento . . . 38
18.5 Consideraciones técnicas . . . 38
18.6 Prioridad sugerida por IC y peso del elemento (HU-067) . . . 38
18.7 Planes rutinarios y ciclos (HU-068) . . . 40
18.8 Evaluación de reglas daño → intervención . . . 40

**19 Foro y notificaciones** **41**
19.1 Descripción . . . 41
19.2 Tecnologías . . . 41
19.3 Funcionamiento . . . 41
19.4 Hilos y notificaciones . . . 41
19.5 Consideraciones técnicas . . . 41

**20 Auditoría** **43**
20.1 Descripción . . . 43
20.2 Tecnologías . . . 43
20.3 Funcionamiento . . . 43
20.4 Registro de auditoría . . . 43
20.5 Consideraciones técnicas . . . 43
20.6 Consulta administrativa y lectura de eventos . . . 45

**21 Despliegue y recuperación** **45**
21.1 Descripción . . . 45
21.2 Tecnologías . . . 45
21.3 Funcionamiento . . . 45

21.4 Despliegue y recuperación . . . . . 46
21.5 Consideraciones técnicas . . . . . 46
21.6 Procesos programados y coordinación . . . . . 46
21.7 Comprobación inicial y diagnóstico . . . . . 47

# 1 Introducción y alcance

## 1.1 Propósito

El Sistema de Gestión de Puentes (SGP) es una aplicación web progresiva para administrar el inventario de puentes y apoyar su inspección técnica, revisión académica, colaboración y mantenimiento. Está dirigido a administradores, catedráticos, estudiantes y profesionales externos, con permisos asociados al rol y al contexto de trabajo.

## 1.2 Alcance funcional

El sistema relaciona puentes y su ubicación territorial con formularios de inspección versionados, evidencias, resultados del índice de condición y acciones de mantenimiento. Para el trabajo de campo dispone de preparación de salidas y persistencia local que permite continuar capturas con conectividad limitada y sincronizarlas posteriormente.

La solución se compone de un cliente Angular PWA, una API REST Spring Boot y PostgreSQL con PostGIS. La API concentra validaciones y reglas de negocio; el cliente presenta las funciones y se comunica con ella. Los capítulos generales describen el contexto, el entorno y las prácticas transversales. Los módulos 01–14 desarrollan la arquitectura y los dominios funcionales.

## 1.3 Límites

El SGP registra información y apoya procesos técnicos y académicos. La publicación de un resultado o una propuesta de mantenimiento depende de las reglas configuradas y de las acciones autorizadas en el sistema. Este manual describe el comportamiento implementado; no establece criterios de ingeniería civil externos ni afirma que un despliegue particular esté activo.

# 2 Tecnologías y dependencias principales

La tabla resume las tecnologías presentes en la configuración del proyecto. Las versiones exactas se fijan en Maven, npm o las imágenes de contenedor según corresponda.

| Capa | Tecnología | Función principal |
|---|---|---|
| Backend | Java 21, Spring Boot 3.3.13 | API REST, lógica de negocio, validación y seguridad. |
| Persistencia | Spring Data JPA, Hibernate Spatial, MapStruct | Acceso ORM, consultas geográficas y mapeo entre DTO y entidades. |
| Contratos | Springdoc OpenAPI, JSON Schema Validator | Descripción de API y validación de esquemas dinámicos. |
| Seguridad | Spring Security, JJWT, BCrypt | Autorización, tokens y protección de credenciales. |
| Base de datos | PostgreSQL 16, PostGIS 3.5, Flyway | Datos relacionales y espaciales, con evolución versionada del esquema. |
| Cliente | Angular 21 (`^21.2.0`), TypeScript 5.9, RxJS 7 | Aplicación web y coordinación de operaciones asíncronas. |

| Capa | Tecnología | Función principal |
|---|---|---|
| Interfaz y mapas | PrimeNG, PrimeIcons, MapLibre GL | Componentes de interfaz y visualización geográfica. |
| PWA y trabajo local | Angular Service Worker, Dexie, IndexedDB | Caché de recursos y persistencia local estructurada. |
| Archivos | AWS SDK for Java S3, Apache PDFBox, ImageIO/WebP | Almacenamiento de objetos, generación/validación de PDF y procesamiento de imágenes. |
| Operación | Docker Compose, Nginx, MinIO, ShedLock | Entorno contenedorizado, proxy, almacenamiento compatible con S3 y coordinación de tareas. |
| Pruebas | JUnit/Spring Test, Testcontainers, Playwright | Pruebas de backend, integración con PostgreSQL y pruebas de navegador. |

El backend se compila con Maven. El cliente utiliza npm y el archivo de bloqueo para instalar las dependencias declaradas. En CI se usa Node.js 24; la imagen de compilación del cliente también utiliza Node 24.

# 3 Requisitos de hardware y software

## 3.1 Software

| Componente | Requisito configurado | Observación |
|---|---|---|
| Backend local | JDK 21 y Maven | No hay Maven Wrapper; la imagen de compilación utiliza Maven 3.9.9. |
| Cliente local | Node.js compatible con Angular; Node 24 recomendado | CI y Docker usan Node 24. npm instala desde `package-lock.json`. |
| Base de datos local | Docker y Docker Compose, o PostgreSQL 16 con PostGIS 3.5 | Compose publica PostgreSQL en `localhost:55432`. |
| Aplicación completa en contenedores | Docker con soporte de Compose | El cliente se sirve por Nginx en `localhost:8088`. |
| Pruebas de integración | Docker accesible al proceso de pruebas | Testcontainers levanta PostgreSQL/PostGIS para pruebas de integración. |
| Navegador | Navegador moderno con soporte para PWA, IndexedDB y service workers | Necesario para las funciones locales del cliente. |

Para ejecutar exclusivamente el cliente, se requieren Node.js y npm. Para iniciar la API localmente

se necesita Java, Maven, una base PostgreSQL/PostGIS accesible y las variables requeridas. Docker simplifica la base local y las pruebas de integración.

### 3.2 Hardware

El proyecto no fija cantidades mínimas de CPU, memoria o almacenamiento para equipos de desarrollo o servidores. Los recursos necesarios dependen del volumen de datos, cantidad de usuarios, imágenes y procesos concurrentes. En desarrollo, el equipo debe disponer de recursos suficientes para las herramientas locales y los contenedores que se ejecuten simultáneamente.

## 4 Instalación y configuración del entorno

Esta guía configura el entorno local con la base de datos en Docker y la API y el cliente ejecutados localmente. Se requieren JDK 21, Maven, Node.js/npm y Docker Compose.

### 4.1 1. Configurar variables

Desde la raíz del proyecto, cree el archivo local de configuración del backend:

```
cp sgp-api/.env.example sgp-api/.env
```

Revise el archivo y establezca credenciales válidas para correo SMTP y administrador inicial. La plantilla configura PostgreSQL local y valores de desarrollo. Sustituya la clave JWT por una cadena aleatoria de al menos 32 caracteres. No comparta ni versiona el archivo `.env`.

Las variables esenciales para iniciar el backend incluyen URL, usuario y contraseña de la base; usuario y contraseña de migración; clave JWT; correo y contraseña SMTP; y credenciales para crear el administrador inicial. `MAIL_HOST`, `MAIL_PORT`, `MAIL_FROM`, CORS y otros parámetros tienen valores predeterminados o pueden ajustarse en el mismo archivo.

Para la duración de sesión, `application.properties` consume `ACCESS_EXPIRATION_TIME_JWT` (900000 ms por defecto) y `REFRESH_EXPIRATION_TIME_JWT` (604800000 ms). El archivo `.env.example` y el README de la API también listan `EXPIRATION_TIME_JWT`, pero `application.properties` no la enlaza; no debe asumirse que esa variable cambia la expiración efectiva.

### 4.2 2. Iniciar PostgreSQL/PostGIS

En la raíz del proyecto:

```
docker compose up -d postgres
docker compose ps
```

Espere a que PostgreSQL aparezca saludable. Compose crea la base `sgp_db`, el usuario de aplicación y el usuario migrador. Los datos locales permanecen en el volumen `postgres_data`.

### 4.3 3. Iniciar la API

En una terminal:

```
cd sgp-api
mvn clean spring-boot:run
```

Flyway aplica las migraciones y Hibernate valida el esquema al iniciar. La API queda en `http://localhost:8090`; compruebe su estado en `/actuator/health`. La documentación OpenAPI está disponible cuando la configuración local permite exponerla.

### 4.4 4. Iniciar Angular

En otra terminal:

```
cd sgp-client
npm ci
npm start
```

Abra `http://localhost:4200`. El servidor de desarrollo reenvía las solicitudes `/api` hacia la API local, por lo que no requiere configurar una URL distinta del backend para este flujo.

### 4.5 Alternativa con Compose

Para construir y ejecutar los servicios de la aplicación definidos en Compose:

```
docker compose up -d --build
docker compose ps
```

La aplicación web queda disponible en `http://localhost:8088`. Para detener contenedores sin borrar datos use `docker compose down`; evite `docker compose down -v` si necesita conservar la base local.

### 4.6 Configuración de archivos

El backend usa almacenamiento local de forma predeterminada. La configuración también admite proveedor S3 con región y bucket, y endpoint explícito para MinIO con credenciales de desarrollo. Asegure que el bucket exista y sea privado cuando se use este proveedor. No se necesita MinIO para iniciar el flujo local predeterminado.

## 5 Base de datos y relaciones principales

La persistencia usa PostgreSQL con PostGIS. Flyway versiona las modificaciones del esquema y Hibernate valida las entidades contra la estructura existente; la aplicación no crea tablas automáticamente. Las relaciones centrales conectan usuarios y cursos con puentes, inspecciones, revisiones y resultados técnicos.

#### 5.0.1 Esquema relacional implementado

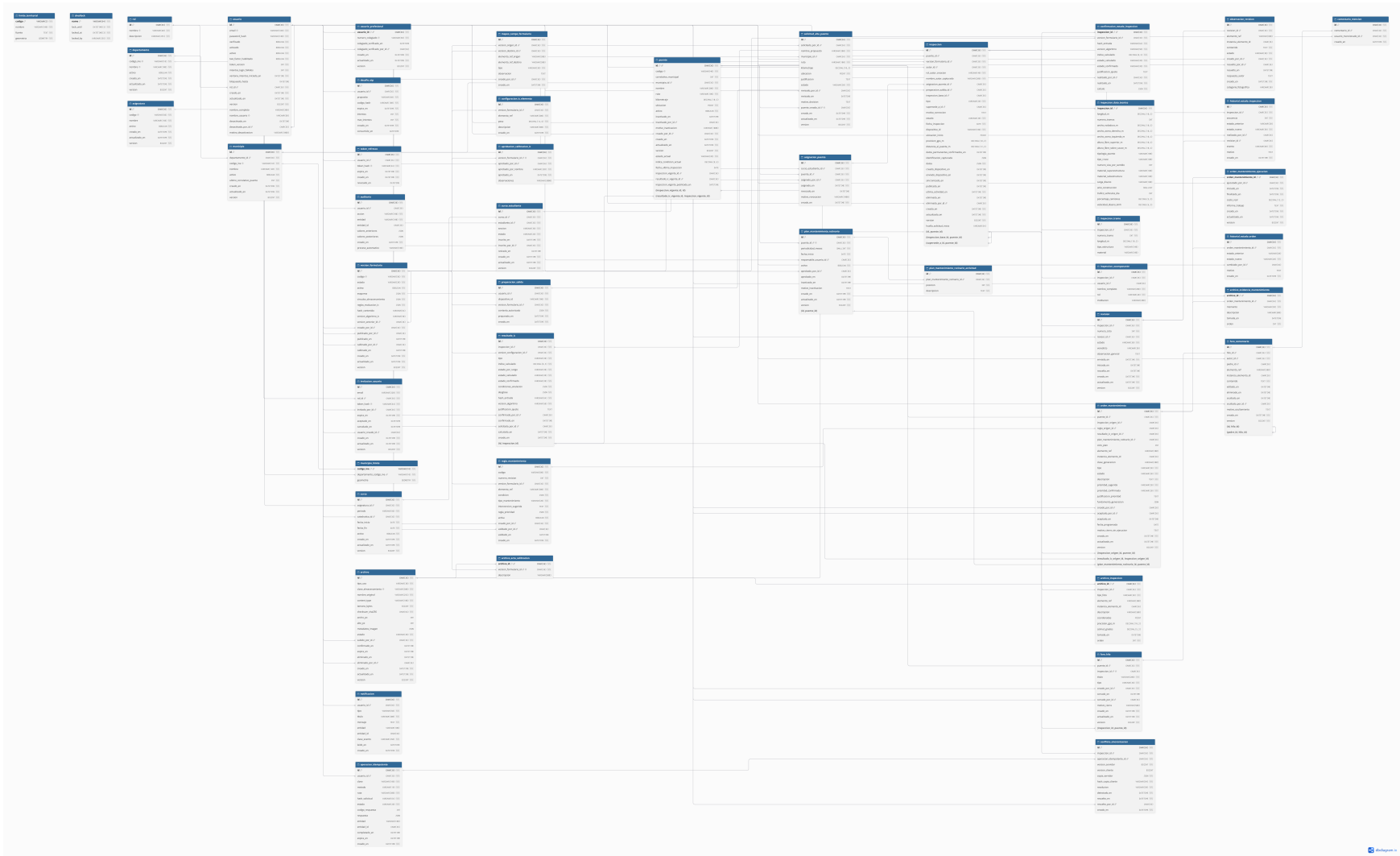


Figura. Esquema general de tablas y relaciones principales de la base de datos.

La imagen presenta el esquema de tablas y sus relaciones a nivel general. El diagrama relacional siguiente resume las entidades principales que intervienen en cursos, asignaciones e inspecciones; ambos sirven como mapas complementarios y no sustituyen las migraciones Flyway, que son la fuente del esquema aplicado.

La relación académica entre cursos, estudiantes y puentes se representa con entidades de asociación, lo que permite conservar sección, estado de inscripción y revocación de asignaciones. Una inspección conserva referencias a su puente, autor y versión de formulario. Puede generar varios resultados IC porque se registran publicaciones y recálculos; los ciclos de revisión se distinguen por número. El diagrama omite catálogos, archivos, bitácoras e historiales para facilitar su lectura.

Los datos espaciales de puentes se almacenan con PostGIS. El contenido variable de formularios y algunos desgloses se guarda en JSONB, mientras las relaciones y restricciones de integridad se expresan con claves foráneas, índices y restricciones de base de datos.

La aplicación genera los identificadores UUIDv7 en Java; las marcas de tiempo se manejan como instantes y Hibernate usa la zona UTC para JDBC. La extensión `pg_trgm` está habilitada y se utiliza en un índice GIN trigram sobre el correo del usuario.

# 6 Estructura del proyecto

El repositorio separa la aplicación en cliente, API, infraestructura y documentación.

| Área | Organización | Responsabilidad |
|---|---|---|
| `sgp-api/` | Backend Spring Boot | API REST y lógica por dominios funcionales. |
| `sgp-client/` | Aplicación Angular PWA | Interfaz, navegación, acceso a API y experiencia offline. |
| `infra/` | PostgreSQL, RDS, staging, producción y respaldos | Inicialización y automatización de operación. |
| `docs/` | Arquitectura, pruebas, base de datos, API y manual técnico | Documentación del sistema y de sus procesos. |
| `.github/workflows/` | Flujos de CI y publicación | Validación automatizada y construcción de imágenes. |

## 6.1 Backend

El código Java se organiza por dominios, entre ellos usuarios, cursos, puentes, formularios, inspecciones, revisión, mantenimiento, archivos, foro, notificaciones y sincronización. Dentro de cada dominio se separan según necesidad los controladores REST, DTO, entidades, repositorios, servicios, mapeadores y validaciones. `common/` reúne capacidades transversales como configuración, seguridad, errores y auditoría.

## 6.2 Cliente

La aplicación organiza pantallas y servicios por `features/`. `core/` aloja navegación y capacidades transversales; `offline/` encapsula la persistencia local y sincronización; `shared/` mantiene

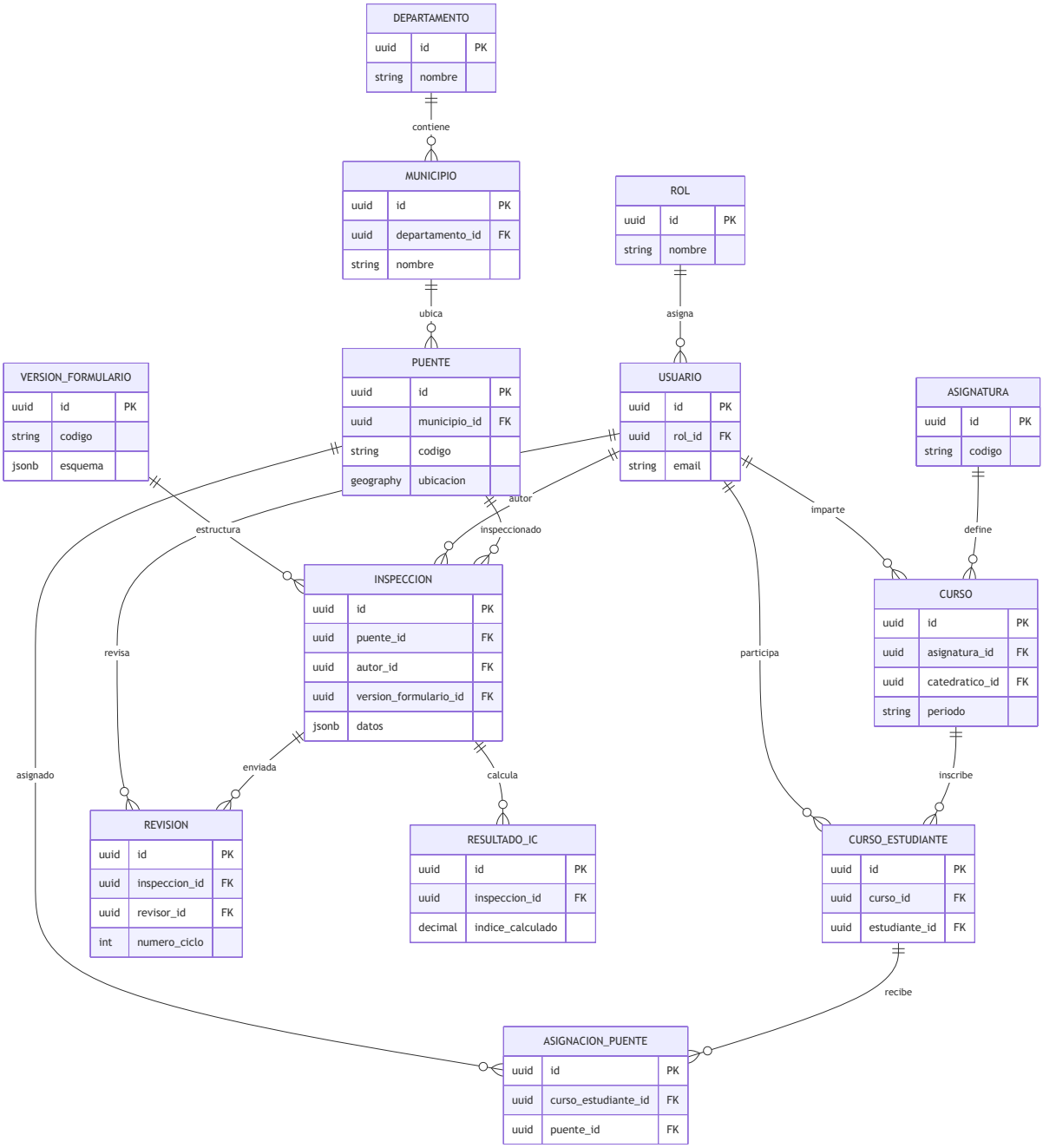



Figura 1: Base de datos y relaciones principales

componentes reutilizables; y `layouts/` y `theme/` contienen la estructura y estilos comunes. Las rutas cargan funcionalidades de forma diferida cuando corresponde.

### 6.2.1 Organización por paquetes

El siguiente esquema relaciona las áreas transversales y funcionalidades del cliente con los dominios del backend. Las funcionalidades usan `core`, componentes compartidos y la capa offline; el backend concentra autenticación, auditoría y manejo de errores como capacidades comunes. Las dependencias entre dominios expresan flujos como inspección y revisión, almacenamiento de evidencias, foro y notificaciones, y sincronización.

## 6.3 Persistencia y documentación

Las migraciones Flyway del backend describen la evolución de PostgreSQL/PostGIS. La documentación API, las guías de prueba y el diccionario de datos acompañan el código. El manual técnico agrupa capítulos generales y módulos funcionales para que puedan consultarse juntos o por separado.

Las decisiones de arquitectura y operación se registran en los ADR del proyecto: 0001, base arquitectónica; 0002, despliegue de staging; 0003, cálculo del IC; 0003, versiones declarativas del formulario SIECA; 0004, despliegue automático de producción; 0005, catálogo provisional de mantenimiento; 0006, propuestas de mantenimiento; 0007, publicación offline; 0007, prioridad sugerida por IC; y 0008, pruebas con red degradada.

# 7 Estrategia y ejecución de pruebas

La validación combina pruebas de lógica, contratos y seguridad con pruebas de integración y verificaciones de construcción. La presencia de una prueba no demuestra que se haya ejecutado ni que haya pasado; los resultados deben consultarse en la ejecución correspondiente.

| Alcance | Herramienta y cobertura | Comando habitual |
|---|---|---|
| Backend | JUnit y Spring Test; pruebas de servicios, controladores y seguridad. | `cd sgp-api && mvn test` |
| Integración backend | Testcontainers con PostgreSQL/PostGIS para persistencia y flujos completos. | `cd sgp-api && mvn verify` |
| Estilo backend | Spotless comprueba formato Java. | `cd sgp-api && mvn spotless:check` |
| Cliente | Pruebas unitarias Angular configuradas por el builder de pruebas del proyecto. | `cd sgp-client && npm test` |
| PWA en navegador | Playwright para escenarios de campo, conectividad y sincronización. | `cd sgp-client && npm run test:pwa` |

| Alcance | Herramienta y cobertura | Comando habitual |
|---|---|---|
| Calidad cliente | ESLint, Prettier y compilación Angular. | `npm run lint`, `npm run format:check`, `npm run build` |

Las pruebas de integración de backend necesitan que Docker esté disponible para que Testcontainers levante la base temporal. La suite Playwright usa un servidor de pruebas propio y concentra los escenarios offline/PWA; algunas fases adicionales se habilitan con variables de entorno definidas por la configuración de pruebas.

Para habilitar una fase específica de HU-062, use el indicador correspondiente antes del comando, por ejemplo `HU062_FASE1=true npm run test:pwa`; también están definidos `HU062_FASE2`, `HU062_FASE3` y `HU062_PAQUETES`.

## 7.1 Verificaciones de CI

El flujo de integración continua ejecuta `mvn -B spotless:check verify` para el backend. Para el cliente instala dependencias desde el archivo de bloqueo y ejecuta lint, formato y build. La configuración del flujo no ejecuta los comandos de pruebas del cliente ni la suite Playwright; deben lanzarse aparte cuando se valide ese alcance.

Al documentar un resultado de prueba, indique el comando, el entorno y su salida real. No infiera aprobación a partir de que el test esté presente o la compilación se haya completado.

## 7.2 Verificación de seguridad

Para autorización, ejecute `cd sgp-api && mvn -Dtest=EndpointsAuthorizationScanTest,AutorizacionNive` `test`. El primero revisa que cada endpoint declare una regla o esté en la lista pública; el segundo comprueba propiedad de borradores y alcance de cursos. Para un controlador concreto, ejecute su clase `*SecurityTest`; los encabezados se prueban con `DeploymentSecurityHeadersTest`. Estos tests no prueban las reglas del proxy ni la configuración del ambiente desplegado. La matriz de roles y la lista de comprobación por ambiente están en Seguridad y sesión.

## 7.3 Pruebas de flujos funcionales

| Pregunta de validación | Prueba de referencia | Qué comprueba y qué no sustituye |
|---|---|---|
| ¿El IC respeta la escala SIECA y los pesos? | `CalculoIcSiecaTest`, `MotorPonderacionIcTest` | Casos de cálculo y ponderación; la integración cubre persistencia/publicación. |
| ¿Se publica junto con el estado vigente del puente? | `PublicacionIndiceCondicionIntegracionTest` | Integración con base real de prueba; necesita Docker/Testcontainers. |

| Pregunta de validación | Prueba de referencia | Qué comprueba y qué no sustituye |
|---|---|---|
| ¿Se conserva el flujo de fotos/documentos? | `FotografiaFlujoIntegrationTest`, `DocumentoInspeccionIntegrationTest` | Persistencia, relación, contenido y limpieza con PostgreSQL y almacenamiento de prueba. |
| ¿Se interpretan los conflictos en el cliente? | `diagnostico-sync.spec.ts`, `diagnostics.page.spec.ts`, `hu061-fase0.integration.spec.ts` | Resumen/visualización local; la prueba e2e HU-062 es la verificación de red y backend integrado. |
| ¿Se evalúa una regla de mantenimiento correctamente? | `EvaluadorCondicionReglaTest`, `PrioridadSugeridaIcTest` | Evaluación de condiciones y fórmula aisladas; no prueba la ejecución de un job del ambiente. |

Ejecute una clase puntual desde `sgp-api` con `mvn -Dtest=NombreDeClase test`. Las pruebas `*IntegrationTest` basadas en PostgreSQL/PostGIS utilizan Testcontainers; compruebe Docker y consulte sus condiciones de ejecución antes de lanzarlas. Para la suite de red degradada, use la guía ejecutable HU-062: especifica sus fixtures desechables, modos, criterios de salida y evidencias, y advierte qué escenarios no certifica. Los nombres de clase indican el alcance previsto, no un resultado aprobado; archive la salida de la ejecución que se esté reportando.

# 8 Arquitectura y modelo de datos

## 8.1 Descripción

El SGP administra el inventario, las inspecciones y el mantenimiento de puentes mediante una aplicación cliente-servidor. El cliente web Angular consume una API REST Spring Boot. PostgreSQL conserva los datos del sistema; PostGIS añade soporte para ubicaciones y consultas geográficas.

## 8.2 Tecnologías

| Componente | Tecnologías relevantes |
|---|---|
| Cliente web | Angular 21, TypeScript, RxJS y Angular Service Worker. Dexie sobre IndexedDB para datos locales offline. |
| API | Java 21, Spring Boot 3.3.13, Spring Web, Spring Security, Spring Data JPA, Spring Validation y MapStruct. Springdoc publica el contrato OpenAPI. |
| Persistencia | PostgreSQL 16, PostGIS y Flyway para las migraciones versionadas. Hibernate accede a los datos y valida su esquema. |

| Componente | Tecnologías relevantes |
|---|---|
| Publicación y archivos | Nginx sirve el cliente y actúa como proxy; Docker Compose integra el entorno local. La API admite almacenamiento local o S3 compatible, incluido MinIO en desarrollo. |

## 8.3 Funcionamiento

El navegador recibe la aplicación Angular desde Nginx. Las pantallas delegan las operaciones a servicios del cliente, que llaman por HTTP los endpoints funcionales bajo `/api/v1`. En el entorno desplegado, Nginx sirve la aplicación y reenvía las solicitudes `/api/` a la API; en desarrollo, el proxy de Angular las dirige al backend local.

La API aplica seguridad y validación en la frontera HTTP, delega las reglas a los servicios de dominio y accede a PostgreSQL mediante repositorios JPA o consultas SQL específicas. Los controladores intercambian DTO con el cliente; las entidades JPA permanecen dentro del backend. Los errores se devuelven en formato `ProblemDetail`, con código y detalles de validación cuando corresponde.

El backend se organiza por dominios funcionales, entre ellos usuarios, cursos, puentes, formularios, inspecciones, revisiones y mantenimiento. Cada dominio contiene las capas que necesita; `common` agrupa capacidades compartidas. El frontend se organiza en funcionalidades, con servicios para las llamadas HTTP y rutas que permiten cargar cada sección de manera diferida. `core` concentra infraestructura compartida y `offline` encapsula el almacenamiento local.

Los datos del servidor se modelan principalmente en tablas relacionales. Las entidades centrales vinculan usuarios, cursos e inscripciones con puentes y asignaciones; los puentes con inspecciones; las inspecciones con versiones de formulario, revisiones y resultados del índice de condición; y las órdenes de mantenimiento con planes, ejecuciones y evidencias. Foro, notificaciones, auditoría e idempotencia respaldan varios flujos.

El esquema también utiliza JSONB para esquemas, respuestas y configuraciones versionadas; tipos geográficos para ubicaciones y límites territoriales; e índices y restricciones para consultas e invariantes de datos. Los archivos binarios se guardan mediante el proveedor configurado, mientras la base de datos conserva sus registros y relaciones.

Flyway aplica las migraciones SQL del proyecto, actualmente desde V1 hasta V25. El navegador también puede conservar recursos de la PWA, algunos datos de referencia y datos de trabajo en IndexedDB. La sincronización offline se comunica con el backend mediante la API; IndexedDB no sustituye a la base de datos del servidor.

## 8.4 Arquitectura de componentes

## 8.5 Consideraciones técnicas

- Flyway gobierna la evolución del esquema. Hibernate tiene `ddl-auto=validate`: comprueba que el mapeo concuerde con la base, pero no crea ni actualiza tablas.
- Los cambios estructurales requieren nuevas migraciones; las ya aplicadas no se modifican.
- JSONB permite conservar estructuras variables, pero las solicitudes siguen sujetas a validaciones de dominio y esquema.

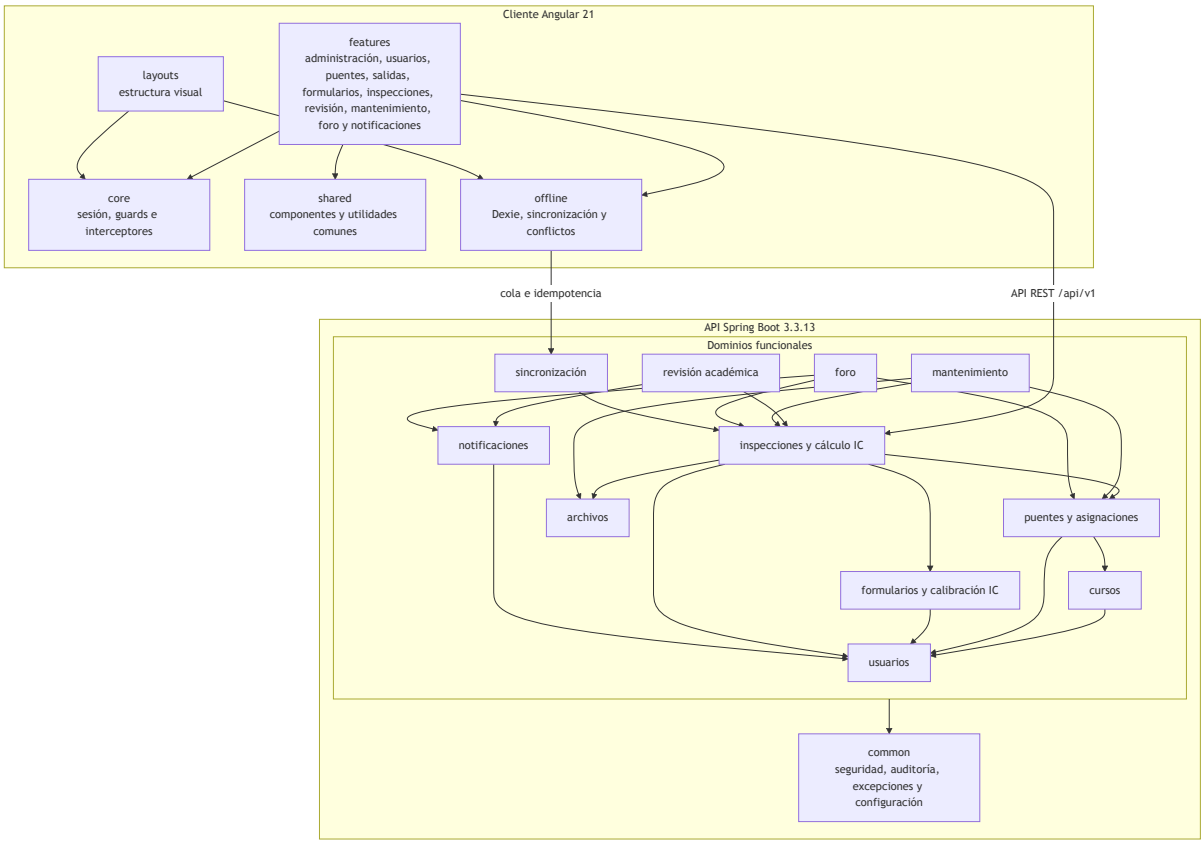



Figura 2: Estructura del proyecto

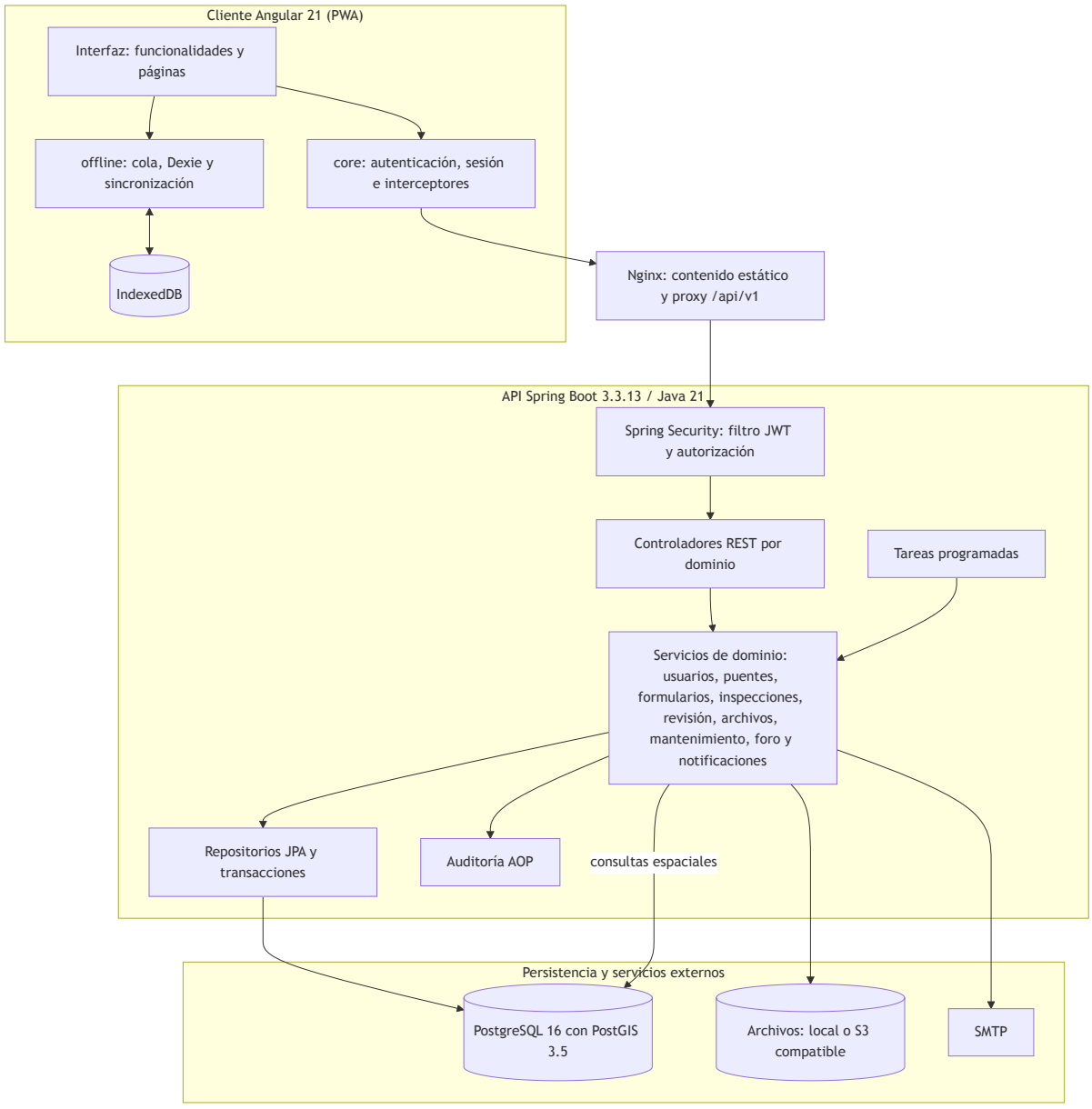



Figura 3: Arquitectura y modelo de datos

- Las ubicaciones y consultas espaciales dependen de PostGIS.
- Los perfiles y variables de ambiente determinan conexiones, seguridad y proveedor de archivos. La configuración disponible no confirma por sí sola qué servicios externos están desplegados.

# 9 Seguridad y sesión

## 9.1 Descripción

Este módulo controla el acceso a la API y mantiene la sesión entre el navegador y el servidor. Incluye registro verificado, autenticación, segundo factor, renovación de credenciales y autorización por roles.

## 9.2 Tecnologías

- Spring Security y JWT con JJWT.
- BCrypt para contraseñas y OTP de un solo uso.
- Angular, RxJS, guards e interceptores HTTP.

## 9.3 Funcionamiento

El registro de estudiantes requiere verificar un código enviado por correo; la cuenta conserva además su estado de activación académica. En el inicio de sesión, la API comprueba la contraseña y el estado de la cuenta. Si el segundo factor está habilitado, solicita un OTP antes de completar la sesión.

La API emite un access token JWT de vida limitada y un refresh token aleatorio. El access token se conserva en memoria en el cliente; el refresh token se entrega en una cookie HttpOnly. La renovación rota el refresh token y revoca el anterior. Ante una respuesta 401, el interceptor intenta renovar la sesión y repetir una vez la solicitud original.

En cada petición protegida, Spring Security valida el JWT y contrasta el estado actual, rol y versión de tokens del usuario. Los guards ayudan a controlar la navegación, pero la autorización efectiva reside en el servidor. La recuperación de contraseña valida un OTP, cambia la contraseña y revoca las sesiones activas.

## 9.4 Flujo de autenticación y renovación

## 9.5 Consideraciones técnicas

- La cookie de refresco usa atributos configurables; `Secure` depende del entorno y `SameSite` tiene un valor predeterminado restrictivo.
- La caché local del perfil permite mostrar contexto durante una desconexión, pero no valida una sesión ante la API.
- Los OTP expiran, limitan intentos y se consumen una sola vez; las rutas públicas y los límites de solicitudes son configurados por el backend.

## 9.6 Autorización por rol y alcance del recurso

Los guards del cliente controlan navegación, pero cada endpoint aplica su propia regla en Spring Security. Esta matriz resume rutas representativas verificadas en los controladores; no reemplaza

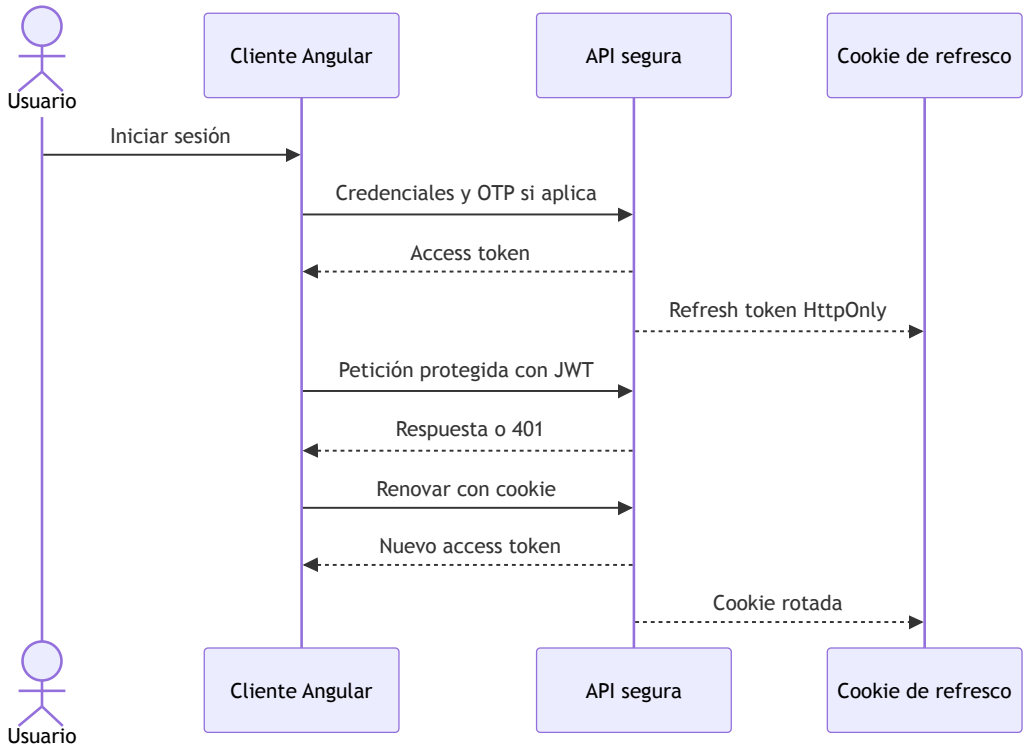



Figura 4: Seguridad y sesión

el inventario completo de `@PreAuthorize`.

| Actor | Acceso representativo | Restricción adicional |
|---|---|---|
| Visitante | Catálogo y ficha de puentes (`GET /api/v1/puentes`, `GET /api/v1/puentes/{id}`) | En estado `Malo`, las coordenadas se ocultan; ver Puentes y asignaciones. |
| Estudiante | Sus propios borradores de inspección | Un borrador ajeno responde 404 para no revelar su existencia. |
| Catedrático | Recursos de los cursos que dirige y revisiones asignadas | La pertenencia al curso se verifica a nivel de recurso. |
| Administrador | Operaciones administrativas, como registrar o modificar puentes | Los endpoints administrativos exigen `ROLE_ADMINISTRADOR`. |

Una petición protegida sin credenciales válidas responde 401; con sesión válida pero rol insuficiente responde 403. Cuando la política oculta la existencia de un recurso ajeno, la respuesta puede ser 404. El código concreto depende de la regla de cada operación; no se debe interpretar un 404 como prueba de que el identificador no existe.

## 9.7 Pruebas y controles por ambiente

`EndpointsAuthorizationScanTest` detecta endpoints sin `@PreAuthorize` ni inclusión explícita en la lista pública. `AutorizacionNivelDeFilaSecurityTest` comprueba propiedad de borradores y acceso a cursos. Las pruebas `*SecurityTest` de cada controlador cubren reglas de rutas concretas; `DeploymentSecurityHeadersTest` verifica encabezados en el contexto de pruebas. La existencia de estos tests no acredita por sí sola un despliegue seguro.

| Control | Evidencia en el repositorio | Estado de staging documentado |
|---|---|---|
| CORS con orígenes explícitos | Configuración de seguridad de la API | Revisar el origen del ambiente al desplegar; la prueba unitaria no sustituye la comprobación del proxy. |
| HTTPS | Nginx y guía de despliegue | HTTPS público fue reportado operativo en la guía de staging. |
| HSTS | Cubierto por pruebas de encabezados | No habilitado en staging según `docs/deployment/STAGING_AWS.md`, por el uso de IP y certificado registrado. |
| Actuator | Configuración de API y redes de contenedores | Documentado como interno; verificar desde fuera del host que el proxy lo rechace. |

| Control | Evidencia en el repositorio | Estado de staging documentado |
|---|---|---|
| Retención central de logs por 90 días | Requisito de HU-073 | No implementada según la guía de staging; no asumir que la rotación local equivale a retención central. |

Antes de liberar cada ambiente, compruebe desde su URL real: redirección HTTP a HTTPS y HSTS; CSP, `X-Content-Type-Options`, `X-Frame-Options` y `Referrer-Policy`; rechazo externo de Actuator; CORS limitado al origen esperado; y ausencia de credenciales o datos personales completos en logs. Registre resultado, fecha y ambiente. La matriz de HU-073 es una guía de verificación: los elementos señalados como faltantes requieren cierre o aceptación explícita de la desviación.

# 10 Usuarios y cursos

## 10.1 Descripción

Este módulo administra las cuentas y los vínculos académicos o profesionales que habilitan la participación en inspecciones y otras tareas del sistema.

## 10.2 Tecnologías

- Spring Boot, Spring Data JPA y Bean Validation.
- PostgreSQL y Flyway.
- Angular y formularios reactivos.

## 10.3 Funcionamiento

Un administrador puede invitar a usuarios con perfil de catedrático o profesional externo. La invitación se asocia a un token aleatorio con vencimiento; el destinatario valida el enlace y completa sus credenciales. Los profesionales registran su número de colegiado, que debe ser verificado por administración antes de habilitar las operaciones que lo requieren.

Los estudiantes se registran y verifican su correo, pero permanecen pendientes de activación. Un catedrático puede activar a un estudiante dentro de un curso activo que tenga asignado, indicando sección. La operación valida que el estudiante esté pendiente y no tenga ya una inscripción activa, y crea su inscripción académica.

Los cursos relacionan período, asignatura y catedrático. Administración puede crearlos, asignar o cambiar responsables y finalizarlos; las inscripciones vinculan estudiantes activos con curso y sección.

## 10.4 Registro e inscripción académica

## 10.5 Consideraciones técnicas

- La invitación no equivale a una sesión activa: el usuario debe completar el proceso de aceptación y establecer sus credenciales.

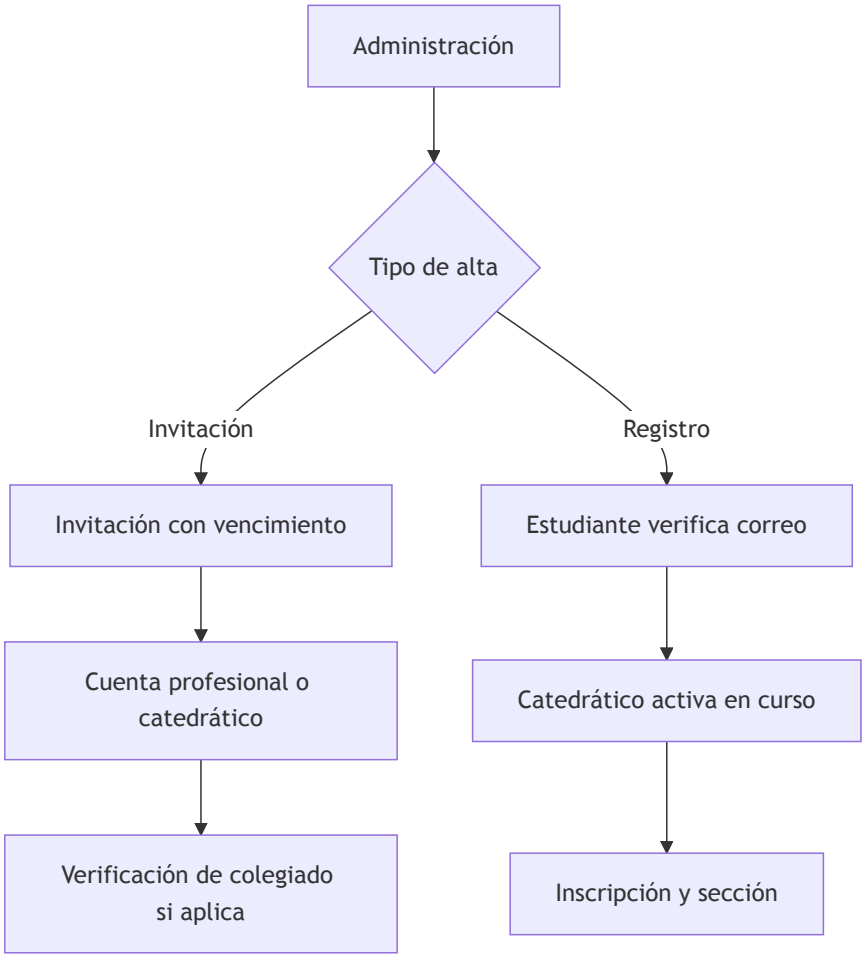



Figura 5: Usuarios y cursos

- La activación estudiantil depende de un curso vigente y de la asignación del catedrático; los roles y estados de cuenta se explican en el módulo de seguridad.
- La vigencia de inscripciones y colegiaturas es relevante para determinar qué operaciones académicas puede realizar cada usuario.

### 10.6 Recorridos de alta y habilitación

| Perfil | Recorrido implementado | Estado que habilita la siguiente acción |
|---|---|---|
| Estudiante | Se registra, verifica el correo y queda pendiente de activación académica. Un Catedrático activo lo vincula a uno de sus cursos activos e indica sección. | La cuenta se activa al crear la inscripción; no basta con verificar el correo. |
| Catedrático | Administración crea una invitación. El usuario valida el enlace vigente y lo acepta para completar la cuenta. | La invitación aceptada crea acceso de cuenta; las operaciones académicas siguen sujetas a las asignaciones del curso. |
| Profesional externo | Administración invita con número de colegiado. El usuario acepta la invitación y Administración verifica después el colegiado. | La publicación directa de inspecciones exige colegiado verificado. |

La activación de estudiantes comprueba que la cuenta esté verificada, pendiente y sin inscripción activa previa; también verifica que el curso pertenezca al Catedrático que realiza la operación. Un curso finalizado no admite nuevas vinculaciones. La activación y la inscripción con sección se registran como una operación auditada. Si el enlace de invitación venció, Administración puede emitir otro mientras la invitación no esté aceptada ni cancelada; el enlace previo queda invalidado. Estos estados explican por qué “cuenta registrada” no significa que todas las funciones estén habilitadas.

## 11 Puentes y asignaciones

### 11.1 Descripción

Este módulo mantiene el inventario de puentes, valida su ubicación y controla las solicitudes de alta y las asignaciones de trabajo a cursos.

### 11.2 Tecnologías

- Spring Boot, Spring Data JPA e Hibernate Spatial.
- PostgreSQL con PostGIS y JTS.
- MapLibre GL en el cliente.

### 11.3 Funcionamiento

El catálogo permite consultar puentes y su ubicación en mapa. Administración puede registrar o modificar fichas; un catedrático puede proponer un alta, que queda pendiente de decisión administrativa. Al guardar una ubicación se validan coordenadas dentro de Guatemala y su pertenencia al municipio seleccionado. Las consultas espaciales detectan puentes cercanos para advertir de posibles duplicados; la proximidad no bloquea automáticamente el alta y puede requerir confirmación.

Las asignaciones vinculan puentes activos con inscripciones académicas vigentes. El sistema impide duplicar una asignación activa y registra la revocación con su motivo. La selección disponible para trabajo de campo se deriva de estas asignaciones y del perfil del usuario.

### 11.4 Inventario y asignaciones

### 11.5 Consideraciones técnicas

- La ubicación se almacena como dato geográfico y se valida espacialmente contra límites municipales; no basta con validar latitud y longitud.
- La alerta de cercanía sirve para revisión de duplicados, no sustituye el criterio de aprobación.
- La revocación afecta la disponibilidad de una asignación para nuevas operaciones; el historial asociado al puente conserva su contexto.

### 11.6 Privacidad de coordenadas para puentes en estado Malo

El catálogo `GET /api/v1/puentes` y la ficha `GET /api/v1/puentes/{id}` son públicos. Si la respuesta corresponde a un puente cuyo estado vigente es `Malo`, el backend oculta latitud y longitud para visitantes. En la ficha también omite las coordenadas UTM. Mantiene el nombre, municipio y estado para identificar el registro; la autenticación permite consultar la ubicación completa. Los demás estados conservan sus coordenadas públicas.

| Solicitud | Coordenadas para `Malo` | Caché HTTP |
|---|---|---|
| Visitante sin sesión | Ocultas en catálogo y ficha | `Cache-Control: private, no-store, max-age=0`; `Vary: Authorization` |
| Usuario autenticado | Incluidas | `Cache-Control: private, no-store, max-age=0`; `Vary: Authorization` |

La política sigue el estado actual del puente, no una copia histórica. `no-store` evita almacenar la respuesta y `Vary: Authorization` separa las variantes por encabezado de autorización. En el catálogo público el puente sigue apareciendo sin ubicación; la capa de mapa no puede dibujar su marcador al no recibir coordenadas. La excepción no cambia la búsqueda cercana usada al registrar puentes. `PuenteCatalogoControllerTest` cubre respuestas públicas/autenticadas y encabezados para la ficha y el catálogo; `PuenteControllerSecurityTest` cubre además autorización de las operaciones de escritura.

## 12 Formularios y calibración

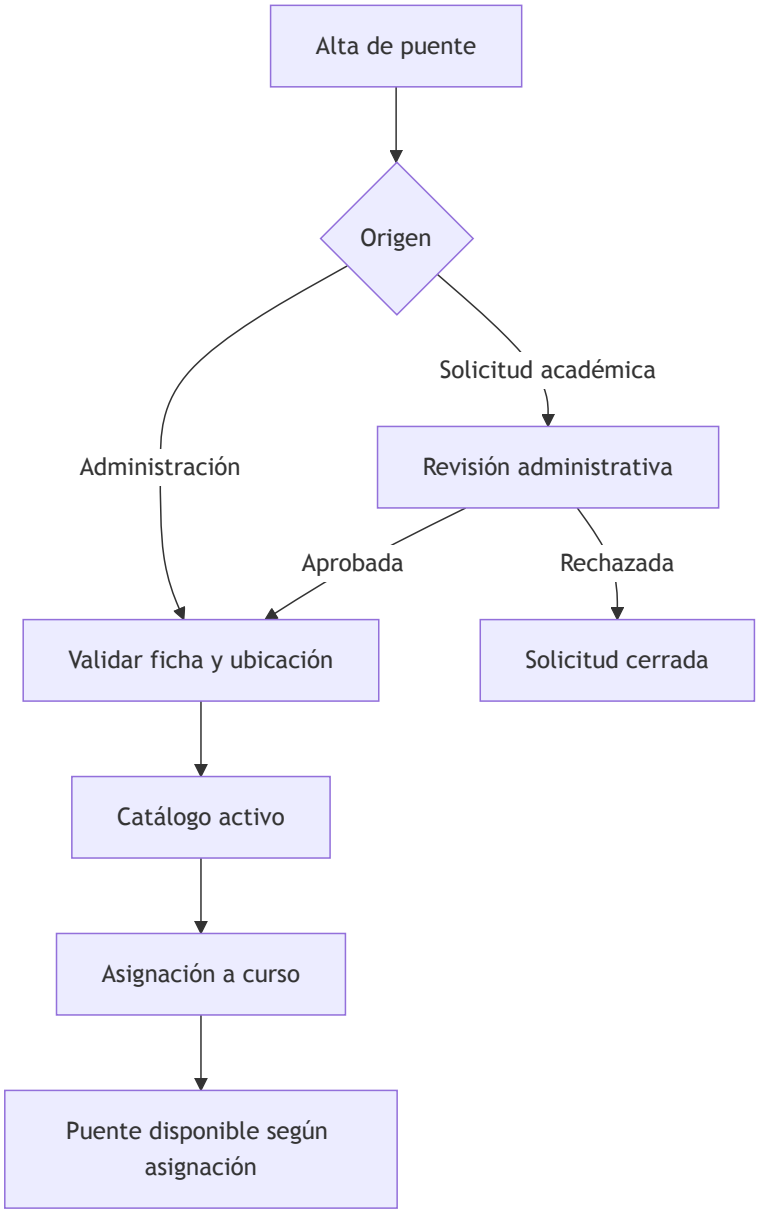



Figura 6: Puentes y asignaciones

## 12.1 Descripción

Este módulo define versiones de formularios técnicos y sus reglas de evaluación. El cliente construye la captura a partir del esquema publicado, mientras la configuración del IC se mantiene asociada a la versión.

## 12.2 Tecnologías

- JSON Schema y validador JSON Schema en Spring.
- Angular Reactive Forms y RxJS.
- PostgreSQL con almacenamiento JSONB.

## 12.3 Funcionamiento

Una versión de formulario se edita como borrador, se valida y luego se publica. El cliente resuelve el esquema y genera controles reactivos, incluidos campos repetibles y reglas de visibilidad condicional. Al guardar, serializa las respuestas visibles y aplica las validaciones derivadas del esquema.

La configuración de evaluación asociada incluye catálogo de elementos, mapeos entre versiones, escala, pesos y reglas críticas. Antes de publicar, el backend verifica que el esquema y la configuración sean coherentes y que los mapeos requeridos estén completos. La publicación calcula una huella del esquema y fija la versión activa; las inspecciones conservan referencia a la versión con la que fueron iniciadas. La calibración aprobada queda asociada a la versión y conserva su acta.

## 12.4 Publicación y calibración de formularios

## 12.5 Consideraciones técnicas

- El esquema de cada versión publicada no se modifica; permite interpretar inspecciones históricas con la estructura original.
- Formularios y reglas se versionan juntos para preservar la relación entre captura y cálculo. El algoritmo del IC se detalla en su propio módulo.
- Las referencias internas del JSON Schema se resuelven con límites de profundidad para evitar recursión indefinida.
- La aprobación de calibración es un proceso separado de la edición del borrador y conserva evidencia documental.

## 12.6 Siembra inicial, publicación y aprobación académica

En el primer arranque, si `app.formulario.siembra.enabled` está habilitado (predeterminado), el inicializador busca una versión activa. Si no existe, requiere que la cuenta de administrador inicial esté disponible y siembra una versión provisional SIECA con código libre `SIECA-vN`, además de su catálogo provisional de reglas. Si ya hay una versión activa, no reemplaza ni crea otra. Un error se registra y el arranque continúa; el log indica que el administrador puede cargar la plantilla desde el editor.

Desde la siembra, la versión sigue el ciclo ordinario: Administrador o Catedrático pueden editarla como borrador; solo Administrador publica y gestiona los mapeos entre versiones. La publicación fija la versión usada por nuevas inspecciones; no altera la estructura histórica de inspecciones

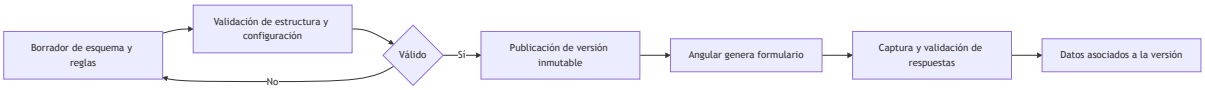



Figura 7: Formularios y calibración

existentes. La aprobación académica ocurre después de publicar y es una acción separada: solo un Catedrático activo puede aprobarla, una sola vez por versión. El acta es un adjunto opcional que Administrador o Catedrático puede consultar o cargar. Al publicarse una versión sin aprobación se avisa a los Catedráticos; al aprobarse, se notifica a los Administradores activos. Por tanto, «publicada» y «aprobada académicamente» son estados distintos y deben comprobarse por separado.

# 13 Índice de condición

## 13.1 Descripción

El módulo calcula el índice de condición (IC) de una inspección y, al publicarla, registra el resultado definitivo y actualiza el estado vigente del puente.

## 13.2 Tecnologías

- Spring Boot, transacciones Spring y Jackson.
- PostgreSQL, JPA y consultas SQL para bloqueo transaccional.
- JSON Schema versionado y datos JSON de inspección.

## 13.3 Funcionamiento

El cálculo toma los datos de inspección junto con el catálogo, los pesos, la escala y las reglas vinculadas a la versión del formulario. Resuelve los elementos evaluables y sus repeticiones, convierte las respuestas a calificaciones, agrega indicadores y aplica las reglas de ponderación y las condiciones críticas. Los elementos excluidos o marcados como no aplicables no participan; los datos requeridos ausentes generan un error de entrada incompleta. El resultado incluye desglose, estado calculado, versión del algoritmo y huella de los datos usados.

Antes de publicar, el sistema vuelve a preparar y calcular el IC definitivo con los datos persistidos. La confirmación permite aceptar el estado calculado o registrar un ajuste justificado cuando las reglas lo permiten. En una sola transacción se guarda el resultado de publicación y se actualiza el estado vigente del puente; un fallo revierte ambos cambios. El recálculo histórico conserva comparaciones y no reemplaza por sí mismo la publicación vigente.

## 13.4 Cálculo y publicación del IC

### 13.4.1 Ejemplo numérico

Para dos elementos incluidos, con calificaciones `2` y `0`, y pesos `2` y `1`, la suma ponderada es `(2 × 2) + (0 × 1) = 4` y la suma de pesos es `3`:

```
IC = 100 × (1 − 4 / (4 × 3)) = 66.666666… → 66.66667
```

El motor usa `BigDecimal` y redondea a cinco decimales con `HALF_UP`; clasifica el valor redondeado. Para la configuración semilla SIECA v1, los límites son BUENO con IC mayor o igual a 75, REGULAR con IC mayor o igual a 50 y MALO por debajo de 50, por lo que el ejemplo queda REGULAR. Cada versión del formulario conserva sus propios límites. Si se activa una condición crítica, el estado calculado pasa a MALO aunque el estado por rango siga siendo REGULAR.

Figura 8: Índice de condición

### 13.5 Consideraciones técnicas

- La configuración asociada a la inspección determina cómo interpretar sus respuestas; cambios en versiones posteriores no alteran el cálculo histórico.
- El valor preliminar sirve para revisión de captura; el definitivo se genera al publicar y se almacena junto con el desglose y la huella.
- El bloqueo del puente evita que publicaciones concurrentes compitan por actualizar su estado vigente.
- Los detalles de estados del ciclo de inspección y de revisión se describen en sus módulos respectivos.

### 13.6 Validaciones del motor y lectura de resultados

El motor rechaza el cálculo si no recibe una versión de formulario, código de versión, versión de algoritmo, hash SHA-256 hexadecimal de 64 caracteres, al menos un elemento evaluado o reglas de rango válidas. Cada elemento debe tener peso positivo, calificación agregada válida y aporte ponderado coherente con `calificación × peso`. La lista de condiciones críticas debe estar presente; cada condición declarada debe producir el estado `MALO`. Estos fallos responden como error de validación 422 en vez de publicar un IC parcial.

Para revisar el desglose, confirme primero que cada fila corresponda a un elemento/instancia de la versión original, después que su aporte sea el producto de calificación y peso y, por último, que la suma ponderada y suma de pesos reproduzcan el índice. El rango se decide sobre el IC redondeado. Con los límites SIECA de la versión semilla, `75` ya es `BUENO`, `50` ya es `REGULAR`, y un valor menor que `50` es `MALO`; otras versiones pueden declarar límites distintos, siempre que cumplan `MALO = 0 < REGULAR < BUENO <= 100`. Una condición crítica cambia solo el estado final a `MALO`: no reemplaza el índice numérico ni la clasificación por rango del desglose.

La prueba `MotorPonderacionIcTest` cubre los límites 0/50/75/100, peso positivo, integridad de aporte, hash y precedencia crítica. La prueba de servicio `CalculoIcSiecaTest` comprueba la resolución de valores desde respuestas del formulario; la integración de publicación verifica la escritura transaccional junto con el puente.

## 14 Archivos y evidencias

### 14.1 Descripción

Este módulo procesa, almacena y vincula fotografías y documentos con inspecciones, calibraciones y órdenes de mantenimiento.

### 14.2 Tecnologías

- ImageIO y TwelveMonkeys ImageIO WebP.
- AWS SDK para S3 y almacenamiento local compatible con el proveedor configurado.
- Apache PDFBox para documentos PDF.
- PostgreSQL para metadatos y relaciones de archivos.

### 14.3 Funcionamiento

Las fotografías se decodifican y normalizan a JPEG RGB; se genera una miniatura y un checksum. La imagen normalizada elimina metadatos EXIF del archivo almacenado, mientras los metadatos

de captura que el flujo reconoce se registran por separado. El servicio guarda los objetos bajo claves privadas y asocia sus metadatos al dominio correspondiente. En inspecciones puede conservarse el anclaje a un elemento, instancia y categoría de evidencia.

La carga reserva primero una operación idempotente, almacena los objetos y luego confirma el registro del archivo y su asociación en la transacción del dominio. Si falla una etapa, intenta eliminar los objetos parciales y compensar la reserva. Los documentos se validan de acuerdo con su tipo de uso y el acceso de descarga se somete a autorización. El proveedor de objetos puede ser local o S3/MinIO según la configuración.

## 14.4 Carga y acceso a evidencias

## 14.5 Consideraciones técnicas

- Se valida el contenido decodificado, no solo la extensión o el MIME declarado; la normalización limita el lado mayor de la imagen a 1600 px y genera miniatura de 300 px.
- En una captura con cámara, el cliente obtiene GPS mediante geolocalización del navegador y registra la hora de captura en UTC. En una imagen de galería exige coordenadas GPS válidas y fecha/hora EXIF con zona, o fecha/hora GPS en UTC; no sustituye esos datos con la ubicación actual del navegador. El servidor valida el anclaje al formulario de la inspección: `elementoRef` (`elemento_ref` en el vínculo persistido) y, si corresponde, la instancia deben pertenecer a esa inspección. Si no se envía referencia, tampoco puede enviarse instancia.
- El cliente conserva temporalmente la imagen procesada y su clave idempotente; la fotografía local solo se elimina tras la confirmación del servidor. Las cargas se reintentan hasta cinco veces y los casos que requieren atención pueden reintentarse desde la interfaz de sincronización (véase el capítulo 09).
- Si falla la confirmación en base de datos después de preparar objetos, el servidor compensa la reserva e intenta eliminar los objetos parciales. Si esa compensación falla, el manifiesto duradero permite recuperar el intento; el archivo temporal local se elimina al terminar la solicitud.
- Cada inspección admite hasta 60 fotografías disponibles y 10 documentos PDF; cada PDF puede medir como máximo 20 MiB. Los documentos se adjuntan y retiran solo mientras la inspección está en borrador.
- Un borrador sin actividad recibe un aviso a los 180 días. A los 210 días se purgan sus archivos multimedia, pero se conserva el registro de la inspección.
- Base de datos y almacenamiento de objetos no comparten una transacción distribuida. Las reservas, compensaciones y procesos de recuperación reducen el riesgo de objetos huérfanos.
- Las claves de almacenamiento no deben tratarse como autorización: el servidor controla el acceso a las descargas.
- Los capítulos de inspección y mantenimiento describen los requisitos de evidencia de cada flujo; aquí se explica el mecanismo común de archivos.

## 14.6 Contrato de carga de una evidencia

Las rutas de inspección requieren autenticación. Para adjuntar una foto se envía `multipart/form-data` a `POST /api/v1/inspecciones/{id}/fotografias`, con las partes `metadatos` (JSON), `archivo` y el encabezado `Idempotency-Key`. El DTO de metadatos contiene `id`, `categoria`, `latitud`, `longitud`, `capturadoEn`, `elementoRef` e `instanciaElementoId`. Las categorías aceptadas son `ACCESO`, `SUPERESTRUCTURA`, `SUBESTRUCTURA`, `CAUCE` y `OTRA`. El identificador y la clave deben

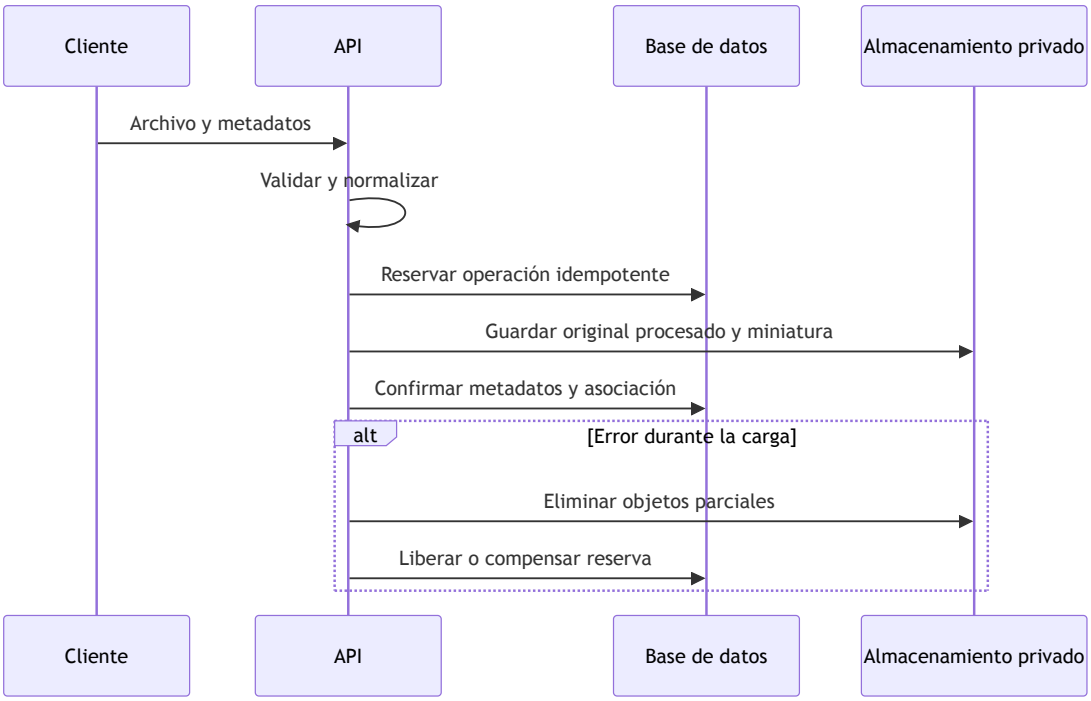



Figura 9: Archivos y evidencias

conservarse en un reintento de la misma solicitud.

Ejemplo esquemático para una foto ya preparada por el cliente:

```
curl -X POST "$API/api/v1/inspecciones/$INSPECCION_ID/fotografias" \\
  -H "Authorization: Bearer $ACCESS_TOKEN" \\
  -H "Idempotency-Key: $CLAVE_UUID" \\
  -F 'metadatos={"id":"'$FOTO_UUID'","categoria":"ACCESO","latitud":14.8,"longitud":-91.5,"ca
  -F 'archivo=@foto.jpg;type=image/jpeg'
```

Para un documento, `POST /api/v1/inspecciones/{id}/documentos` recibe la parte `archivo`; no requiere `Idempotency-Key`. Una respuesta 201 incluye el identificador, nombre original de presentación, tamaño y datos de carga, pero nunca la clave interna del objeto. El contenido se descarga por `GET /api/v1/inspecciones/{id}/documentos/{documentoId}/contenido`, que vuelve a comprobar autorización y envía `no-store` y `nosniff`.

| Respuesta relevante en carga de PDF | Interpretación y acción |
|---|---|
| 401 | Falta sesión válida; renovar o iniciar sesión y repetir la carga. |
| 403 / 409 | El actor no puede modificar esa inspección o ya no está en borrador; no repetir como si fuera un error de transporte. |
| 413 | El archivo supera 20 MiB. |
| 422 | Firma no-PDF o límite de 10 documentos. Corregir el archivo o retirar otro mientras el borrador sea editable. |
| 503 | Almacenamiento no disponible; no se registra el documento y puede reintentarse cuando vuelva el servicio. |

La retirada de un adjunto es un cambio de asociación disponible solo durante el borrador. Después del envío o publicación los documentos quedan fijos. En fotos, el endpoint de miniatura y el de contenido son lecturas autorizadas; la URL por sí sola no concede acceso. El cliente elimina su copia pendiente únicamente al recibir confirmación, así que un corte después de subir bytes pero antes de recibir la respuesta debe resolverse con la misma clave y el mismo UUID, no creando una carga nueva.

# 15 Inspecciones

## 15.1 Descripción

Este módulo coordina el ciclo de vida de una inspección: inicio, captura y guardado de respuestas, envío, publicación directa o revisión académica.

## 15.2 Tecnologías

- Spring Boot, Spring Data JPA y Bean Validation.
- Angular y formularios reactivos generados desde esquemas.
- PostgreSQL con JSONB y control transaccional.

## 15.3 Funcionamiento

Al iniciar, la inspección queda asociada a un puente, a su autor y a una versión concreta del formulario. La captura puede guardarse como borrador y se valida contra esa versión. Antes del envío o publicación se comprueban la completitud de respuestas y las evidencias requeridas para el flujo.

Las transiciones se validan como eventos desde un estado actual y se registran en el historial. Las inspecciones de estudiantes se envían a un ciclo de revisión de un catedrático; las de catedráticos y profesionales externos verificados pueden publicarse directamente. El revisor puede aprobar, solicitar cambios o rechazar, con motivo escrito para las dos últimas decisiones. Una corrección vuelve al autor para edición según el estado permitido. La publicación integra el IC definitivo en la transacción y emite un evento para procesos posteriores.

## 15.4 Ciclo de vida de la inspección

## 15.5 Consideraciones técnicas

- La máquina de estados rechaza eventos incompatibles y los estados terminales no admiten nuevas transiciones.
- La concurrencia se controla al modificar la inspección; una solicitud incompatible puede devolver conflicto en lugar de sobrescribir cambios.
- Publicación, historial, IC definitivo y estado vigente del puente se confirman juntos. Las notificaciones y generación de mantenimiento ocurren en etapas posteriores.
- La sincronización offline y los detalles del cálculo, revisión y almacenamiento de evidencias se explican en sus capítulos propios.

## 15.6 Consulta histórica, impresión y exportación (HU-072)

El detalle de una inspección publicada interpreta las respuestas con el esquema de la versión de formulario asociada al registro; no sustituye esa estructura por la versión activa. La vista identifica la versión de origen y, cuando hay mapeo, muestra su relación con la activa. El acceso a la inspección y sus archivos requiere sesión.

Desde el detalle se puede imprimir con el diálogo del navegador (`window.print()`). La hoja de estilos de impresión organiza la ficha, respuestas, reporte de evidencias y propuestas de mantenimiento. La exportación descarga JSON sanitizado desde `GET /api/v1/inspecciones/{id}/exportar`; incluye datos y esquema de la versión histórica, resumen del puente, IC disponible, metadatos de fotografías y propuestas. No incluye documentos adjuntos en la estructura JSON; estos se consultan por separado desde el detalle. La API limita la exportación a inspecciones publicadas y responde 404 si no encuentra una publicada con ese identificador.

Para validar una inspección antigua, compare el código y esquema incluidos en el JSON con la versión mostrada en la ficha; revise luego impresión y archivos por separado. `InspeccionServiceHu072Test` cubre la exportación sanitizada y el caso inexistente; `detalle-publicado.page.spec.ts` verifica la acción de exportar desde el cliente.

## 15.7 Secuencia de captura y cierre

La API crea la inspección como `BORRADOR` y fija el puente, el autor y la versión del formulario en ese momento. La solicitud de inicio incluye fecha, `dispositivoId` y `puenteId`; si el inspector

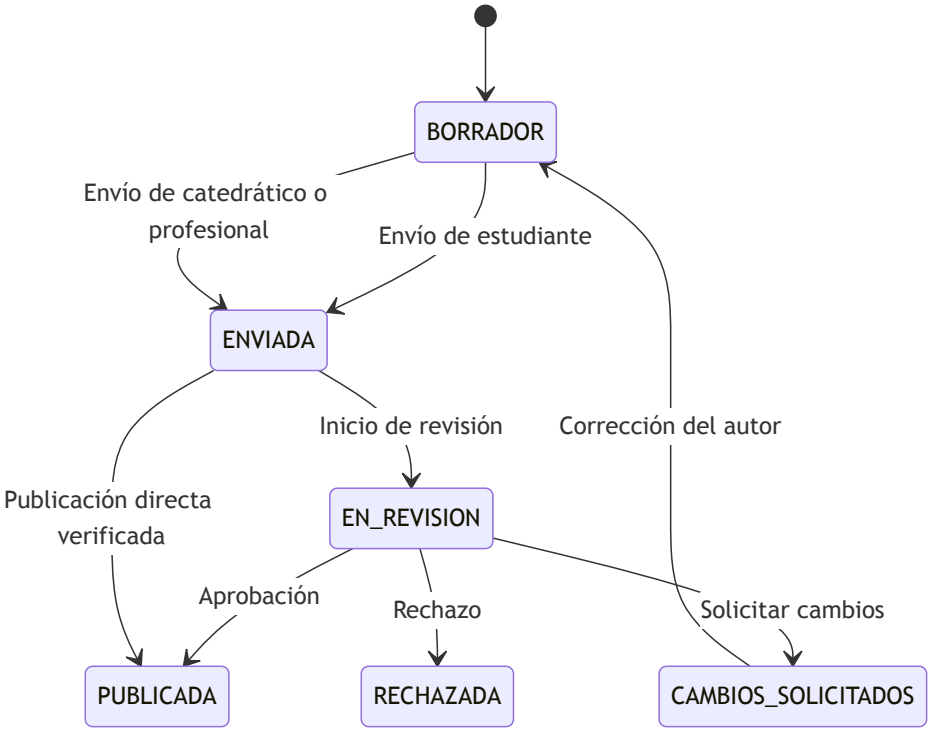



Figura 10: Inspecciones

confirma explícitamente la precarga, también se copian los datos técnicos permanentes de la última inspección publicada. La respuesta devuelve el identificador nuevo, el estado, el código de la versión y cualquier advertencia de ubicación GPS. Si la misma creación se reintenta con la misma clave de idempotencia, se reutiliza el resultado y no se crea otra inspección.

Una captura se guarda como borrador de manera parcial; la publicación o envío, en cambio, vuelve a validar el documento completo y las evidencias requeridas. El flujo de una cuenta de Estudiante es:

1. Iniciar una inspección para un puente asignado y guardar respuestas parciales.
2. Adjuntar las fotos necesarias para accesos, superestructura, subestructura y cauce; la validación informa categorías faltantes.
3. Enviar desde `POST /api/v1/inspecciones/{id}/enviar`. Si falta un campo o una foto requerida, la API responde 422 y mantiene el borrador corregible.
4. El Catedrático asignado revisa y aprueba, solicita cambios o rechaza. Una solicitud de cambios devuelve la inspección al autor; la nueva corrección conserva el historial del ciclo anterior.
5. Con la aprobación se publica la inspección y su IC definitivo.

Un Catedrático o Profesional externo verificado usa `POST /api/v1/inspecciones/{id}/publicar`; no pasa por la revisión académica y la operación verifica el colegiado del profesional cuando aplica. Una cuenta de Estudiante recibe 403 si intenta esa publicación directa. Las transiciones no compatibles con el estado actual responden 409. En ambas rutas válidas, la escritura de la publicación, el historial, el resultado del IC y el estado vigente del puente se confirman conjuntamente; una falla en esa transacción no deja una publicación parcial.

Después del commit se notifican los flujos dependientes. La creación de propuestas de mantenimiento se ejecuta en una transacción separada, por lo que su error no revierte la publicación; un Administrador puede regenerar las propuestas faltantes. Este límite transaccional ayuda a diagnosticar un caso donde la inspección aparece publicada pero aún no tiene órdenes asociadas.

## 15.8 Contratos de API para el recorrido

| Paso | Endpoint | Resultado observable |
|---|---|---|
| Crear | `POST /api/v1/inspecciones` | 201, `Location` con la nueva inspección y respuesta en `BORRADOR`. El request identifica puente, fecha y dispositivo; la versión activa se fija si no se indica una versión publicada específica. |
| Consultar/editar borrador | `GET /api/v1/inspecciones/{id}/borrador`, `PUT /api/v1/inspecciones/{id}/borrador` | El autor recupera o guarda respuestas y versión optimista. Una versión desactualizada se rechaza con 409 en vez de sobrescribir. |

| Paso | Endpoint | Resultado observable |
|---|---|---|
| Enviar estudiante | `POST /api/v1/inspecciones/{id}/enviar` | Requiere rol Estudiante, autoría, borrador completo y evidencias. Acepta `Idempotency-Key`; respuesta 200 confirma la transición a revisión. |
| Publicar sin revisión | `POST /api/v1/inspecciones/{id}/publicar` | Requiere Catedrático o Profesional externo verificado que sea autor; respuesta 200 al completar publicación e IC. |
| Exportar historial | `GET /api/v1/inspecciones/{id}/exportar` | Devuelve JSON solo para una inspección publicada; véase HU-072 arriba. |

Las operaciones de consulta/edición de borrador están protegidas por sesión y verifican el autor; cuando un usuario ajeno solicita un borrador, el servicio responde 404 para no revelar su existencia. 401 indica falta de sesión válida, 403 un rol/acción no autorizados, 409 una versión o transición incompatible y 422 una captura incompleta o inválida. El contrato exacto de errores es `ProblemDetail` con código de dominio; el detalle de evidencias faltantes incluye categoría y cantidad requerida/disponible. No reintente un 409 con una clave nueva sin recargar el borrador: primero concilie la versión local y la del servidor según el capítulo offline.

# 16 Offline y sincronización

## 16.1 Descripción

Este módulo permite preparar salidas de campo, conservar inspecciones y fotografías en el dispositivo y sincronizar las operaciones cuando vuelve la conectividad.

## 16.2 Tecnologías

- Angular Service Worker (PWA) y Dexie sobre IndexedDB.
- API REST de sincronización e idempotencia en Spring.
- Almacenamiento del navegador y señales de conectividad.

## 16.3 Funcionamiento

La API prepara un paquete de trabajo según el usuario, los puentes autorizados y la versión activa del formulario. El cliente guarda el paquete y los datos de campo localmente mediante la capa offline. IndexedDB conserva entidades pendientes y blobs de fotografía; el service worker administra la caché de recursos de la aplicación.

Cuando hay conexión, el sincronizador procesa las operaciones pendientes en orden: crea o actualiza el borrador, carga las fotografías y luego envía o publica la inspección. Cada operación conserva una clave idempotente para repetir solicitudes sin duplicar el efecto. La API vincula la clave con el usuario, método, ruta y huella de solicitud; si recibe la misma operación

completada reproduce su respuesta, y rechaza reutilizarla con contenido distinto. Los conflictos de concurrencia quedan identificados para diagnóstico y resolución.

## 16.4 Captura y sincronización sin conexión

## 16.5 Consideraciones técnicas

- La pantalla de diagnóstico de sincronización muestra operaciones pendientes, intentos y errores, y permite reintentar o sincronizar. También consulta el espacio usado/disponible y el estado de persistencia del almacenamiento del navegador; la opción para solicitar persistencia usa la API del navegador y su concesión no está garantizada.
- Los fallos transitorios de red y HTTP 5xx se reintentan con espera creciente hasta cinco intentos por ciclo automático. Si una carga de fotografía agota los intentos o falla por un error no reintentable, pasa a `INTERVENCION`. Un 401 deja la operación pendiente y reinicia el contador; restablezca la sesión del mismo usuario y sincronice. La pantalla muestra los intentos y permite pulsar **Reintentar**, que reinicia el contador, o **Sincronizar ahora** después de recuperar la conexión.
- Al reanudar la aplicación, una operación `ENVIANDO` sin actividad por más de 60 segundos se recupera como `PENDIENTE`, salvo que ya haya agotado los intentos, caso en que requiere intervención. La reanudación periódica y los eventos de conexión o inicio de sesión funcionan con la aplicación abierta; no se garantiza sincronización con ella cerrada.
- Si el servidor rechaza definitivamente el envío o la publicación, la intención queda `RECHAZADA` y el borrador local se conserva para corregirlo y confirmar de nuevo. Mantenga la sesión del propietario original: las colas están separadas por usuario y no se transfieren al cambiar de cuenta.
- Los datos locales se separan por propietario; cambiar de usuario no debe sincronizar pendientes de otra cuenta.
- Reintentar con la misma clave es seguro solo si se conserva la misma solicitud. La idempotencia no resuelve conflictos de edición concurrente.
- La cuota disponible depende del navegador y dispositivo; la estimación de almacenamiento no garantiza que futuras escrituras tengan éxito.
- El service worker cachea recursos de la app; no reemplaza IndexedDB como persistencia de los registros de campo.

## 16.6 Divergencias y copia del servidor (HU-059)

Cuando la API detecta que la inspección del servidor cambió respecto de la versión de campo, conserva el trabajo local como versión principal de la sincronización y guarda una instantánea separada de la versión del servidor en `conflicto_sincronizacion`. La vista de diagnóstico muestra ambas copias; la del servidor es de solo lectura. El sistema no mezcla campos automáticamente ni ofrece aplicar la copia remota sobre la local: el inspector compara ambas y, si necesita corregir su trabajo, edita la copia local mediante el flujo normal. La instantánea no reemplaza las respuestas del inspector.

El conflicto queda asociado a la inspección, autor, fecha y versión del servidor. Repetir la misma operación con su clave idempotente no debe duplicar el registro, y otro usuario no puede consultar esa copia. Esta resolución conserva datos para revisión; no implica que toda la sincronización sea aceptada ni omite validaciones de estado y negocio del servidor.

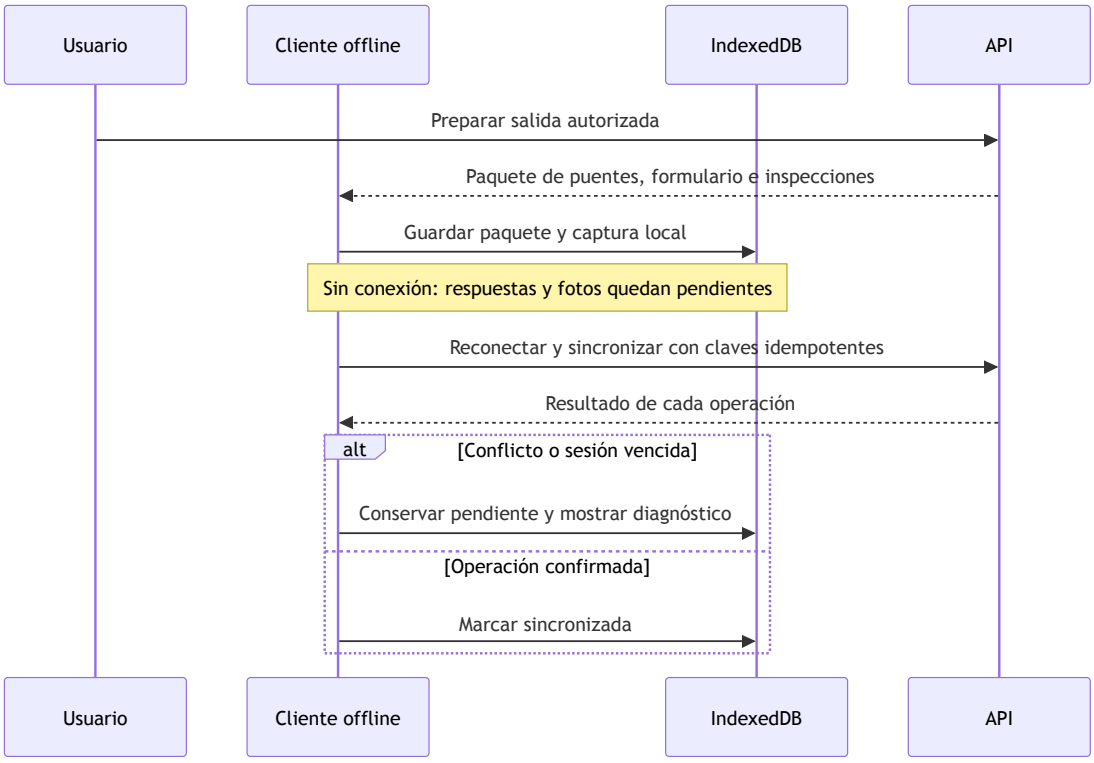



Figura 11: Offline y sincronización

# 17 Revisión académica

## 17.1 Descripción

Este módulo gestiona la revisión de inspecciones estudiantiles, sus ciclos, observaciones y resolución por parte del catedrático.

## 17.2 Tecnologías

- Spring Boot, Spring Data JPA y transacciones.
- Angular para bandejas, detalle y atención de observaciones.
- PostgreSQL para ciclos, observaciones e historial.

## 17.3 Funcionamiento

Al enviar una inspección estudiantil se crea un ciclo de revisión asignado al catedrático correspondiente. El revisor consulta la bandeja, inicia el ciclo y revisa respuestas y evidencias. Puede anotar un elemento o instancia del formulario, una categoría fotográfica o una observación general. El backend valida que los anclajes existan en la versión del formulario y que la categoría corresponda al contexto.

El estudiante atiende las observaciones y remite la corrección; el revisor verifica su resolución y emite el veredicto del ciclo. La aprobación publica la inspección y coordina el cálculo definitivo del IC; solicitar cambios devuelve la inspección al autor y el rechazo la cierra. Los ciclos y estados se mantienen para conservar la trazabilidad de revisiones sucesivas.

## 17.4 Ciclo de revisión académica

## 17.5 Consideraciones técnicas

- Solo el catedrático asignado puede resolver su ciclo; el autor y revisor deben ser distintos.
- La revisión se vincula a una versión concreta del formulario, por lo que los anclajes no dependen de la versión activa más reciente.
- La aprobación sigue las validaciones de completitud y evidencia de publicación; el detalle del IC definitivo está en su módulo.
- Las observaciones y correcciones se conservan por ciclo, evitando que una nueva revisión sobrescriba la anterior.

## 17.6 Resultado de un ciclo y diagnóstico

| Decisión del Catedrático | Efecto sobre inspección | Siguiente responsable |
|---|---|---|
| Aprobar | Publica con IC definitivo y actualiza el estado vigente del puente. | Procesos posteriores reciben el evento de publicación. |
| Solicitar cambios | Registra motivo y observaciones; deja la inspección disponible para corrección del autor. | Estudiante atiende las observaciones y reenvía. |
| Rechazar | Registra el motivo y termina la inspección sin publicarla. | No hay corrección posterior de ese ciclo. |

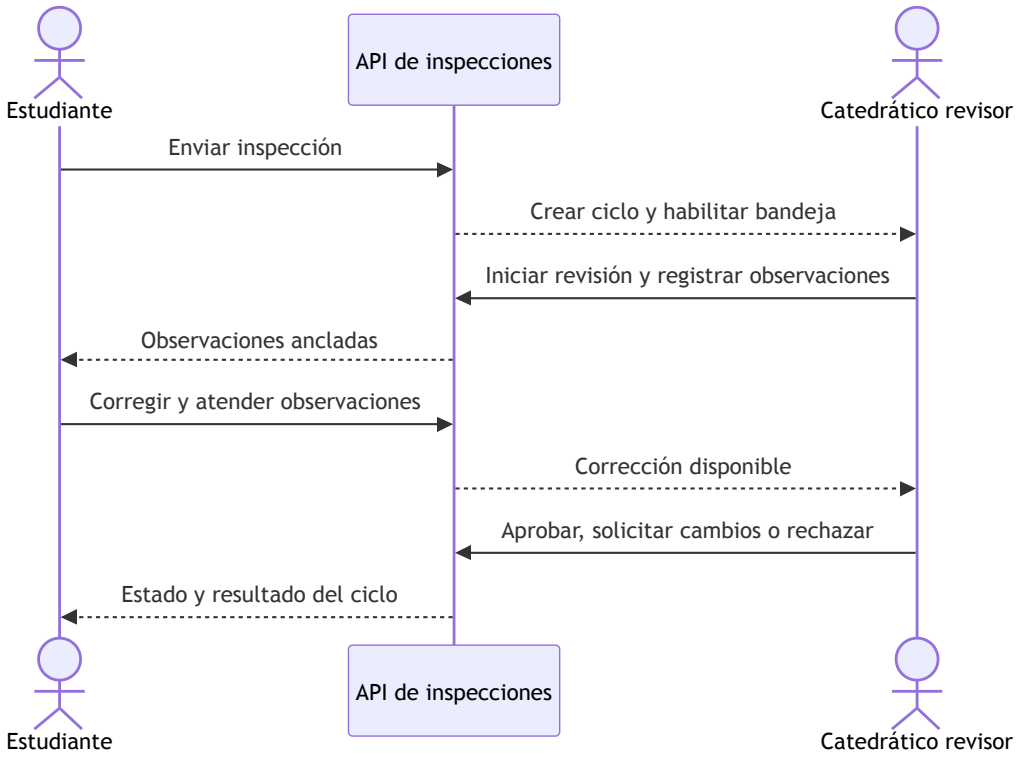



Figura 12: Revisión académica

El diagnóstico de una revisión debe seguir la secuencia de estados y los registros del ciclo, no limitarse a la bandeja actual: una inspección que volvió a corrección ya no aparece necesariamente como pendiente del mismo paso. Una observación puede referirse a una ruta del esquema y a una instancia repetida; por eso, al cambiar de versión de formulario, se conserva la versión de captura y el anclaje se valida contra ella. La publicación aprobada comparte las validaciones de respuestas obligatorias y evidencias con la publicación directa.

# 18 Mantenimiento

## 18.1 Descripción

Este módulo genera propuestas de mantenimiento desde inspecciones publicadas, administra planes rutinarios y controla órdenes hasta su ejecución.

## 18.2 Tecnologías

- Spring Boot, Spring Data JPA y eventos transaccionales.
- PostgreSQL y JSONB para reglas y resultados.
- ShedLock para coordinar tareas programadas.

## 18.3 Funcionamiento

Después de confirmarse la publicación de una inspección, un listener evalúa las reglas activas de la versión del formulario usada. Las condiciones se contrastan con los elementos inspeccionados y el resultado del IC; cada coincidencia puede generar una orden en estado `PROPUESTA`. La generación es idempotente y evita volver a proponer una orden equivalente que siga en curso. Administración puede regenerar propuestas faltantes.

Las propuestas no se aceptan ni programan automáticamente. Un usuario con permisos puede aceptarlas y programarlas, descartarlas con motivo, iniciar su ejecución o cancelarlas con motivo. El cierre como ejecutada exige datos de ejecución y evidencia. En paralelo, los planes rutinarios generan ciclos vencidos y órdenes asociadas; una tarea programada coordina ese trabajo con bloqueo distribuido.

## 18.4 Generación y ejecución de mantenimiento

## 18.5 Consideraciones técnicas

- La generación automática ocurre después del commit de la publicación y en una transacción independiente: un error de mantenimiento no revierte una inspección ya publicada.
- Las órdenes generadas permanecen como propuestas hasta una decisión operativa; la prioridad sugerida y la evaluación de reglas no equivalen a programación.
- Los estados terminales no admiten nuevas transiciones; cambios de estado conservan historial.
- Las tareas programadas usan ShedLock para evitar ejecución concurrente entre instancias.

## 18.6 Prioridad sugerida por IC y peso del elemento (HU-067)

Al generar una propuesta desde una inspección publicada, el sistema combina la prioridad base de la regla con el IC publicado y el peso calibrado del elemento en la versión exacta del formulario. La




Figura 13: Mantenimiento

fórmula de ADR 0007 normaliza el peso contra el mayor peso positivo configurado en esa versión:

```
D = (100 - IC) / 100
W = peso del elemento / máximo peso positivo de la versión
índice sugerido = índice base + roundHalfUp((3 - índice base) × D × W)
```

Los índices son `BAJA=0`, `MEDIA=1`, `ALTA=2` y `CRITICA=3`; `roundHalfUp` redondea los empates hacia arriba. Por ejemplo, una regla con base `MEDIA`, IC 40, peso 4 y máximo de versión 8 produce `D=0.60`, `W=0.50` y `roundHalfUp((3-1)×0.60×0.50)=1`; la sugerencia es `ALTA`.

Si falta el IC o el peso, hay una inconsistencia de versión, o alguno de los valores está fuera de rango, se conserva la prioridad base y el fundamento indica el motivo; una propuesta afectada no impide generar las demás. Una prioridad base ausente o ilegible mantiene el fallback existente `MEDIA`. La prioridad **sugerida** es inmutable; la **confirmada** puede cambiarse por Catedrático o Administrador con justificación obligatoria y auditoría. No se puede cambiar en órdenes `EJECUTADA`, `CANCELADA` o `DESCARTADA`. Las propuestas históricas no se recalculan.

## 18.7 Planes rutinarios y ciclos (HU-068)

Cada puente activo cuenta con un plan, creado por defecto para puentes nuevos y reconciliado para los activos anteriores sin duplicarlos. El plan permite definir periodicidad de 1 a 12 meses y actividades. Al vencer un ciclo, el proceso crea una orden `PROPUESTA` identificada por el ciclo; la clave del plan y número de ciclo evitan duplicados aun si ya existe una orden en curso o ejecutada. La fecha se calcula en `America/Guatemala`. Un puente inactivo no genera ciclos nuevos y conserva el historial.

El proceso también propone ciclos vencidos que aún no tienen orden: si una ejecución estuvo detenida y encuentra varios períodos vencidos, los recorre y genera los faltantes. Una orden sigue siendo una propuesta y requiere decisión operativa. La configuración del plan requiere permisos de Catedrático o Administrador; estudiantes y profesionales externos solo pueden consultarlo. La periodicidad inválida devuelve 422 y no actualiza el plan.

## 18.8 Evaluación de reglas daño → intervención

El catálogo está versionado junto con el formulario. Una regla identifica `elemento_ref`, un indicador, operador, umbral, tipo de intervención y prioridad base. Una edición crea otra revisión de la regla en lugar de cambiar la que ya generó propuestas. Administración crea/edita reglas; el Catedrático realiza la validación académica. Las reglas provisionales activas pueden operar antes de esa validación y deben leerse como parámetros pendientes de respaldo académico, no como una aprobación implícita.

La condición se evalúa dentro de cada elemento resuelto, una vez por instancia si el campo se repite. Se admiten `IGUAL`, `DISTINTO`, `MAYOR_O_IGUAL`, `MENOR_O_IGUAL` y `CONTIENE`; las comparaciones de orden aceptan números o la escala `NO_PRESENTA < LEVE < MODERADO < SEVERO`. Un valor ausente no se interpreta como cero ni `false`. Por ejemplo:

```
{"indicador":"exposicion_acero","operador":"MAYOR_O_IGUAL","umbral":"MODERADO"}
```

Si el elemento inspeccionado contiene `{"exposicion_acero":"SEVERO"}`, la condición se cumple y puede producir una propuesta; `LEVE` no la cumple. El evaluador devuelve el valor observado para que el fundamento de generación conserve qué dato activó la regla. `MAYOR_O_IGUAL` sobre texto solo ordena valores de la escala conocida; un texto ajeno a ella no se compara.

Una condición incompleta, un indicador sin respuesta o una comparación incompatible no dispara esa regla; una excepción aislada incrementa `reglasConError` y no detiene las demás. La respuesta de regeneración separa `reglasEvaluadas`, `creadas`, `yaGeneradas`, `omitidasPorOrdenAbierta` y `reglasConError`, lo que permite distinguir "no coincidió" de "ya existía" y "falló la evaluación". Para una inspección publicada que quedó sin propuestas, Administración puede ejecutar `POST /api/v1/inspecciones/{id}/propuestas-mantenimiento/regenerar`; requiere rol Administrador, responde 409 si no está publicada y es idempotente. La base conserva la clave de generación y los snapshots de la regla usada para que una regla modificada después no cambie el fundamento histórico.

Pruebas de referencia: `EvaluadorCondicionReglaTest` cubre operadores, escala y datos ausentes; `GeneracionPropuestasMantenimientoIntegrationTest` comprueba generación y no duplicación; `PrioridadSugeridaIcTest` prueba el ajuste de prioridad.

# 19 Foro y notificaciones

## 19.1 Descripción

Este módulo habilita la colaboración en torno a puentes e inspecciones publicadas y entrega avisos internos sobre eventos relevantes del sistema.

## 19.2 Tecnologías

- Spring Boot, Spring Data JPA y eventos transaccionales.
- PostgreSQL para hilos, comentarios y notificaciones.
- Angular para consulta, contadores y avisos offline.

## 19.3 Funcionamiento

El foro organiza hilos permanentes por puente y un hilo asociado a cada inspección publicada. Los comentarios permiten respuestas de un nivel; en el hilo de inspección pueden anclarse a un campo o elemento del formulario. La lectura es pública en los hilos disponibles, mientras que participar requiere sesión. Catedráticos y administradores pueden moderar comentarios y cerrar hilos; se aplican límites de participación.

La publicación de una inspección crea su hilo después de confirmar la transacción. Los comentarios y otras acciones del sistema pueden producir notificaciones internas para usuarios destinatarios. Cada aviso incluye una clave de evento para evitar duplicados y puede consultarse, marcarse como leído o resumirse por usuario. El cliente actualiza los avisos mediante consultas periódicas y conserva contexto de notificaciones para uso offline.

## 19.4 Hilos y notificaciones

## 19.5 Consideraciones técnicas

- Los hilos de puente se clasifican como `DISCUSION_TECNICA`, `PROPUESTA_INTERVENCION`, `CONSULTA`, `REFERENCIA_DOCUMENTAL` o `ALERTA`. Una mención `@usuario` avisa una sola vez por comentario al usuario existente y activo; los nombres inexistentes o desactivados se ignoran.

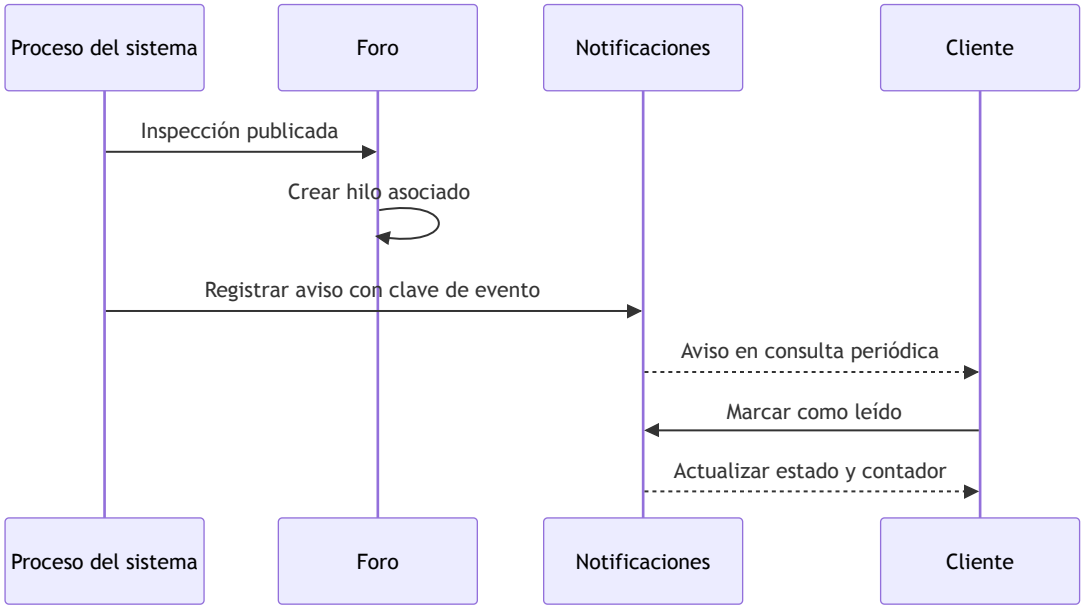



Figura 14: Foro y notificaciones

- Un hilo `ALERTA` de un puente cuyo estado actual es `Malo` genera avisos para catedráticos y administradores activos, excepto quien abrió el hilo. La notificación no incluye el texto publicado.
- Catedráticos y administradores pueden ocultar comentarios y cerrar hilos. Ambas acciones requieren un motivo y registran la moderación en la auditoría; sin motivo la API rechaza la solicitud.
- El hilo asociado a una inspección se habilita solo después de su publicación; su creación ocurre después del commit.
- La validez de un anclaje depende del esquema de la inspección, no del formulario activo actual.
- La clave de evento evita duplicar avisos para el mismo usuario y evento; los textos también se validan para no incluir patrones de credenciales o calificaciones.
- Los avisos de la aplicación son persistentes e internos. Los correos de autenticación e invitación pertenecen a sus flujos respectivos.

# 20 Auditoría

## 20.1 Descripción

La auditoría conserva cambios relevantes de negocio con actor, entidad, fecha y valores antes y después, y permite consultar la trazabilidad desde las herramientas administrativas.

## 20.2 Tecnologías

- Spring AOP y anotaciones de auditoría.
- Spring Transactions y Spring Data JPA.
- Jackson para representar snapshots en JSON.

## 20.3 Funcionamiento

Para operaciones anotadas, un aspecto obtiene el estado anterior mediante proveedores de snapshot, ejecuta la operación y captura el estado posterior. También existen escrituras explícitas desde servicios cuando una operación requiere registrar auditoría dentro de la misma transacción de negocio. Los procesos programados pueden registrar ejecuciones sin actor humano. La consulta administrativa presenta los registros y sus diferencias.

La escritura puede participar en la transacción existente o ejecutarse en una transacción independiente, según el tipo de registro. Los cambios que requieren atomicidad usan la transacción actual; otros registros se escriben con una frontera transaccional propia. Los logs operativos son un canal distinto y no reemplazan la bitácora persistente.

## 20.4 Registro de auditoría

## 20.5 Consideraciones técnicas

- La captura anterior/posterior depende de proveedores que proyectan datos del dominio; los snapshots no deben exponer secretos o información innecesaria.
- La propagación transaccional define si el registro acompaña al cambio o sobrevive a su rollback. Las llamadas explícitas deben respetar esa intención.

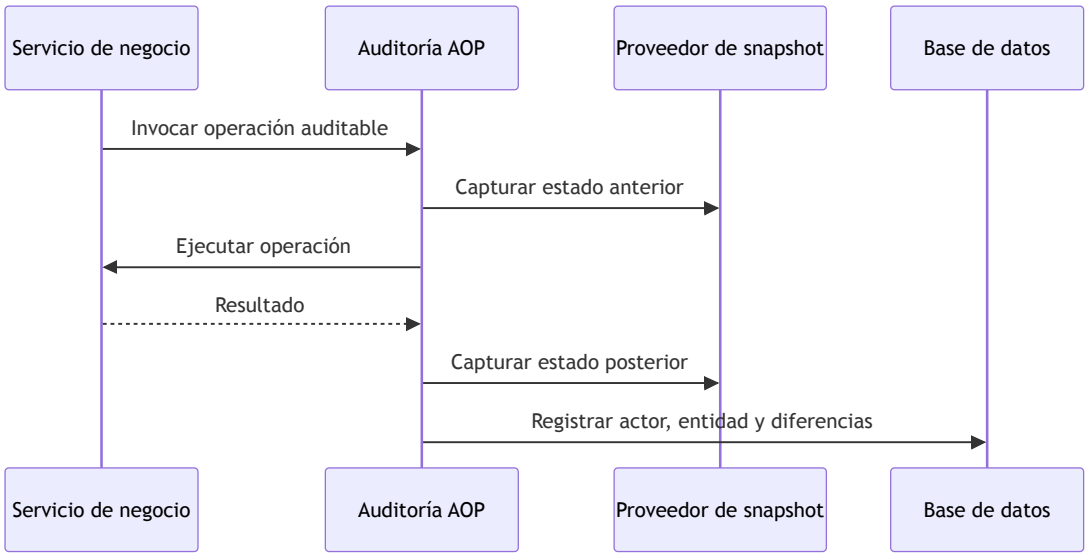



Figura 15: Auditoría

- Las fallas de captura se registran en logs y pueden dejar el snapshot incompleto; los logs no constituyen un reemplazo de la auditoría consultable.
- Los registros de tareas automáticas identifican el proceso y su resumen sin atribuirlo a un usuario.

### 20.6 Consulta administrativa y lectura de eventos

La bitácora se consulta mediante `GET /api/v1/auditoria` y su detalle mediante `GET /api/v1/auditoria/{id}`; ambos requieren rol Administrador. El listado acepta filtros por usuario ID o correo, entidad, acción y fechas ISO-8601 con zona; ordena de más reciente a más antiguo y pagina desde cero con tamaño máximo de 100. Un rango inicial posterior al final o un tamaño fuera de límite responde 422. El correo desconocido produce una página vacía, no una consulta sin filtro.

Ejemplo para localizar ejecuciones de trabajos programados en un período:

```
curl -G "$API/api/v1/auditoria" \\
  -H "Authorization: Bearer $ACCESS_TOKEN" \\
  --data-urlencode "entidad=proceso_programado" \\
  --data-urlencode "accion=EJECUTAR_PROCESO" \\
  --data-urlencode "desde=2026-10-27T00:00:00-06:00" \\
  --data-urlencode "hasta=2026-10-29T00:00:00-06:00" \\
  --data-urlencode "pagina=0" --data-urlencode "tamanio=20"
```

En un registro de proceso automático, `procesoAutomatico` identifica el job y `valoresPosteriores` contiene contadores o resumen de ejecución; el actor humano puede ser nulo. En un cambio de negocio, el detalle presenta campos modificados como pares de valor anterior y posterior. Si falta una diferencia, revise también los logs del servicio: la captura de snapshots puede fallar y dejar la auditoría parcial. Las pruebas `AuditoriaConsultaServiceTest`, `AuditoriaAspectTest` y `AuditoriaControllerSecurityTest` cubren filtros/paginación, captura AOP y autorización.

## 21 Despliegue y recuperación

### 21.1 Descripción

Este módulo describe la ejecución del cliente y la API junto con PostgreSQL/PostGIS, el manejo de configuración por entorno, el despliegue automatizado y la recuperación de datos.

### 21.2 Tecnologías

- Java 21, Spring Boot y Maven.
- Angular, Node.js y Nginx.
- Docker Compose, PostgreSQL con PostGIS y Flyway.
- AWS S3 para objetos y respaldos en los perfiles de despliegue que lo configuran.

### 21.3 Funcionamiento

Las imágenes de producción se construyen por separado para API y cliente. Nginx sirve la aplicación web y enruta las solicitudes API al backend; la API se conecta a PostgreSQL y al

almacenamiento de archivos. Las variables y secretos se inyectan en ejecución. Flyway aplica las migraciones al iniciar la API mediante una cuenta de migración distinta de la cuenta de aplicación.

Los scripts de despliegue de staging y producción validan las imágenes candidatas y esperan health checks. El flujo de producción conserva la configuración anterior y puede restaurarla automáticamente si el despliegue falla. Los scripts de respaldo generan volcados de PostgreSQL y los almacenan en S3; el procedimiento de prueba restaura un respaldo en una base temporal y consulta migraciones y datos para comprobarlo.

## 21.4 Despliegue y recuperación

## 21.5 Consideraciones técnicas

- El respaldo PostgreSQL diario se ejecuta a las 03:00 de Guatemala (09:00 UTC); los volcados diarios se conservan 30 días. El día 1 de cada mes se guarda además una copia mensual, conservada 365 días. La retención automática de RDS es un mecanismo complementario y, en el entorno documentado, está limitada a un día.
- La prueba de restauración usa `infra/backup/restore-test.sh`: restaura el volcado más reciente en una base temporal `sgp_restore_test_<fecha>`, comprueba migraciones, PostGIS y conteos de usuarios, puentes e inspecciones y elimina la base temporal al terminar. Los pasos operativos están en la guía de respaldo y restauración.
- Los perfiles de ejecución requieren configurar URL y cuentas de base de datos, clave JWT, CORS, correo y almacenamiento; los valores de ejemplo no son secretos válidos.
- La cuenta de migración requiere permisos de esquema distintos de la cuenta de aplicación. Las migraciones aplicadas son parte del historial operativo.
- El rollback del despliegue restaura imágenes y composición previa; no revierte cambios de esquema de base de datos.
- La existencia de scripts de AWS, S3 o staging describe capacidad configurada en el proyecto y no demuestra que un ambiente esté desplegado o que una restauración haya sido ejecutada.
- La recuperación debe validarse periódicamente con restauraciones de prueba, no solo con la existencia de un archivo de respaldo.

## 21.6 Procesos programados y coordinación

Los procesos periódicos de la API usan ShedLock para que varias instancias no ejecuten simultáneamente el mismo trabajo. El conjunto diario usa por defecto las 02:00 de `America/Guatemala`; el generador de ciclos rutinarios usa un intervalo fijo de 24 horas por defecto, con su propio bloqueo. La frecuencia puede cambiarse mediante propiedades de aplicación.

| Proceso | Frecuencia predeterminada | Efecto operativo |
|---|---|---|
| Revisiones sin atender | Diario, 02:00 Guatemala | Avisa al revisor asignado cuando una inspección lleva 15 días sin revisión. |

| Proceso | Frecuencia predeterminada | Efecto operativo |
|---|---|---|
| Puentes y órdenes | Diario, 02:00 Guatemala | Avisa por puentes activos sin inspección publicada en 12 meses y por órdenes próximas/vencidas; las alertas no cambian el estado de las órdenes. |
| Ciclos rutinarios | Cada 24 h desde la ejecución anterior | Crea planes faltantes y propuestas para ciclos vencidos; no duplica ciclos existentes. |
| Retención de borradores | Diario, 02:00 Guatemala | Advierte al alcanzar 180 días y elimina multimedia al alcanzar 210 días; conserva el registro del borrador. |
| Limpieza de claves idempotentes | Diario, 02:00 Guatemala | Elimina claves vencidas según su retención configurada. |
| Recuperación de cargas pendientes | Cada 60 s | Revisa cargas que quedaron en estado de envío para permitir su recuperación. |

Los procesos de cron diario usan `app.procesos.cron-diario` (predeterminado `0 0 2 * * *`) y zona `America/Guatemala`; su bloqueo tiene duración mínima de 5 minutos y máxima de 30. El generador de ciclos usa `app.mantenimiento.ciclos-intervalo-ms` (86 400 000 ms) y bloqueo máximo de 10 minutos; la recuperación de cargas usa `app.archivos.recuperacion-intervalo-ms` (60 000 ms), también con bloqueo máximo de 10 minutos. El bloqueo distribuido evita concurrencia entre nodos, mientras las claves idempotentes evitan repetir efectos de negocio. Estos jobs son independientes de la copia diaria de base de datos a las 03:00; el respaldo y su retención están descritos arriba y en la guía de respaldo.

## 21.7 Comprobación inicial y diagnóstico

Después de iniciar el stack local, primero separe un problema de base de datos de uno de aplicación o de proxy. Compose espera que PostgreSQL pase `pg_isready` antes de iniciar el backend; la API ejecuta Flyway y valida el esquema al arrancar. Para revisar el entorno local:

```
docker compose ps
docker compose logs --tail=100 postgres backend nginx
curl --fail --show-error http://localhost:8090/actuator/health
curl --head --fail --show-error http://localhost:8088/
```

La solicitud directa a `:8090` aplica cuando se ejecuta la API desde el host (`mvn spring-boot:run`); el Compose raíz no publica el puerto del backend y sirve la aplicación por Nginx en `:8088`. No interprete el resultado de una configuración local como prueba de acceso externo al Actuator de staging.

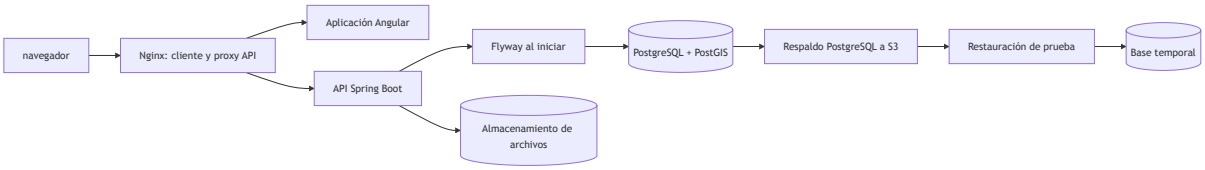



Figura 16: Despliegue y recuperación

| Síntoma | Comprobación siguiente |
|---|---|
| PostgreSQL no está `healthy` | Revisar `docker compose logs postgres`, el estado del contenedor y que el volumen local pueda escribirse. Flyway no puede iniciar antes que la base. |
| Backend termina al arrancar | Revisar el primer error de `docker compose logs backend`: conexión/credenciales, cuenta migradora, validación Hibernate o migración fallida. No edite una migración que ya se aplicó para corregir el despliegue. |
| API está saludable, pero no carga el cliente | Comprobar `nginx`, el proxy `/api/` y la URL del navegador; separar un error de ruta estática de una respuesta API. |
| Carga de archivo responde 503 | Comprobar proveedor `app.archivos.proveedor`, bucket/endpoint y credenciales configurados, además de conectividad y permisos. El mensaje no debe llevar a repetir la carga con un identificador nuevo si la primera respuesta fue incierta. |

Si una versión nueva requiere revertirse, el procedimiento de despliegue puede volver a la composición e imágenes anteriores; no revierte Flyway. Antes de arrancar una imagen anterior contra el esquema actual, compruebe compatibilidad de lectura/escritura. Para recuperación de datos, use la prueba de restauración en base temporal y el procedimiento de la guía enlazada, nunca una restauración experimental sobre la base activa.